



**BAR-ILAN UNIVERSITY**

# TR-PTQ: Log-Domain Integer-Only Post- Training Quantization of Transformers via Taylor- Region Reformulation

Eliyahu Levy

Submitted in partial fulfillment of the requirements for  
the Master's Degree in the Faculty of Engineering

Bar Ilan University

Ramat-Gan, Israel

25/02/2026

This Work was carried out under the supervision of  
Professor Adam Teman

Faculty of Engineering

Bar-Ilan University



# Acknowledgments

I would like to thank all the members of the EnICS Labs for sharing their knowledge and support throughout this work, and especially Udi Kra, who first introduced me to the field of hardware-oriented neural networks, which became a central part of my studies. I am grateful to my family for always being there when I needed them. Above all, I thank Adi, a great mentor and teacher, who guided me through the research process and helped me grow academically. Finally, I thank Keren, who often pushed me further than I thought I could go. Thank you all.

---

# Abstract

Transformer architectures have become a central component of modern machine learning systems, achieving state-of-the-art performance across both vision and language tasks. However, their computational complexity and memory bandwidth requirements pose significant challenges for deployment on edge platforms and custom hardware accelerators with limited silicon area and power budgets. Post-training quantization (PTQ) provides an efficient deployment strategy by converting pretrained floating-point models into low-precision representations without retraining. While PTQ has been successfully applied to convolutional networks, transformer models remain considerably more sensitive to quantization, particularly under strict integer-only execution constraints required for fixed-point hardware implementations.

Existing approaches commonly attribute the accuracy degradation in quantized transformers to limited numerical precision in nonlinear operations such as SoftMax, Gaussian Error Linear Unit (GELU), and layer normalization (LayerNorm). Consequently, many deployment frameworks retain floating-point arithmetic or iterative refinement schemes for these components, increasing hardware complexity, silicon area, and latency variability. In this work, we demonstrate that performance degradation is driven primarily by structural error amplification mechanisms rather than bit-width limitations alone.

Our analysis identifies two dominant contributors. First, learned scale parameters in normalization layers amplify quantization noise due to inter-channel variance heterogeneity and dynamic range expansion. Second, compounded approximation errors in GELU accumulate when nonlinear operators are independently approximated in fixed-point arithmetic. In contrast, SoftMax is observed to remain relatively robust under aggressive quantization when numerical stabilization is preserved. These findings indicate that the principal challenge lies in the structural formulation of normalization and nonlinear interactions rather than in exponential evaluation itself.

To address these challenges, we propose **TR-PTQ**, an integer-only post-training quantization framework based on a Taylor Region (TR) reformulation of nonlinear functions in the logarithmic domain. By expressing multiplicative, divisive, and square-root interactions in log-space, complex operations are reduced to additions and constant scaling

---

operations in integer arithmetic. This transformation eliminates the need for dedicated division and square-root hardware units and avoids iterative numerical methods, while maintaining controlled approximation error. Nonlinear functions are implemented using shared polynomial approximations and compact lookup-table-assisted primitives, yielding a unified, deterministic integer datapath.

The proposed framework consolidates exponential and logarithmic building blocks across SoftMax, GELU, and LayerNorm, reducing hardware redundancy and removing floating-point fallbacks for nonlinear layers. Additionally, we introduce an outlier-aware optimization of LayerNorm parameters during calibration, mitigating dynamic range amplification without retraining or mixed-precision execution.

We evaluate TR-PTQ on representative transformer models for both vision and language tasks under 8-bit quantization. Experimental results show that the proposed method maintains accuracy within approximately 1.5% absolute degradation compared to full-precision baselines across standard benchmarks. These results demonstrate that a log-domain structural reformulation aligned with hardware constraints enables stable and fully integer-only transformer inference without iterative computation or floating-point support.

A manuscript based on this work has been submitted to the International Conference on Machine Learning (ICML) and is currently under peer review.



# Contents

|          |                                                                                     |           |
|----------|-------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                 | <b>1</b>  |
| <b>2</b> | <b>Background</b>                                                                   | <b>5</b>  |
| 2.1      | Transformer overview . . . . .                                                      | 5         |
| 2.1.1    | Multi-Head-Self-Attention (MHSA) . . . . .                                          | 5         |
| 2.2      | Vision Transformers (ViTs) . . . . .                                                | 6         |
| 2.3      | Quantization . . . . .                                                              | 8         |
| 2.3.1    | Uniform Quantization . . . . .                                                      | 9         |
| 2.3.2    | Re-quantization . . . . .                                                           | 9         |
| 2.3.3    | Integer-only Quantization . . . . .                                                 | 11        |
| 2.3.4    | Limitations of prior works . . . . .                                                | 16        |
| <b>3</b> | <b>Efficient Nonlinear Computation for Quantized Transformer Models</b>             | <b>19</b> |
| 3.1      | Integer-only Taylor Region Approximation for Nonlinear Functions . . . . .          | 19        |
| 3.1.1    | Taylor Region-Based Approximation of the Exponential Function<br>(TR-exp) . . . . . | 19        |
| 3.1.2    | Shift-based Taylor Region Natural Logarithm Approximation (TR-ln)                   | 24        |
| 3.2      | Hardware-Efficient TR Calculation of Nonlinear Deep Learning Operators .            | 26        |
| 3.2.1    | Taylor Region SoftMax Approximation (TR-SoftMax) . . . . .                          | 26        |
| 3.2.2    | Taylor Region GELU Approximation (TR-GELU) . . . . .                                | 27        |
| 3.2.3    | Taylor Region Layer Normalization Approximation (TR-Norm) . .                       | 29        |
| 3.2.4    | Gamma-like distribution of learned scales . . . . .                                 | 32        |
| 3.2.5    | Distribution-preserving scale optimization . . . . .                                | 32        |
| <b>4</b> | <b>Experiments and Evaluation</b>                                                   | <b>35</b> |
| 4.1      | Implementation Details . . . . .                                                    | 35        |
| 4.1.1    | Quantization Configuration . . . . .                                                | 35        |
| 4.1.2    | Core Error Analysis . . . . .                                                       | 36        |
| 4.1.3    | Accuracy Evaluation . . . . .                                                       | 36        |
| 4.2      | Ablation Study . . . . .                                                            | 37        |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| 4.2.1    | SoftMax Robustness . . . . .           | 37        |
| 4.2.2    | GELU Optimization . . . . .            | 41        |
| 4.2.3    | Layer Normalization Analysis . . . . . | 42        |
| <b>5</b> | <b>Summary and Conclusions</b>         | <b>47</b> |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | High-level illustration of the Vision Transformer. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 6  |
| 2.2 | An example of an image divided into patches, each containing a $3 \times 3$ grid of smaller cells. . . . .                                                                                                                                                                                                                                                                                                                                                                                                    | 7  |
| 2.3 | Statistical distribution of variance across layers in the <b>bert-uncased-whole-word-masking</b> model evaluated on the SQuAD dataset. . . . .                                                                                                                                                                                                                                                                                                                                                                | 14 |
| 3.1 | Taylor Regions for $e^x$ . Vertical dotted lines denote anchor points; dashed curves show local Taylor approximations centered at each anchor. . . . .                                                                                                                                                                                                                                                                                                                                                        | 21 |
| 3.2 | Block-level overview of TR-exp. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 22 |
| 3.3 | Block-level overview of TR-ln. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 26 |
| 3.4 | Block-level overview of the proposed TR-Norm (top) and TR-GELU (bottom) operators, illustrating their integration within the low-precision non-linear processing pipeline. . . . .                                                                                                                                                                                                                                                                                                                            | 30 |
| 3.5 | Effect of relaxation on LayerNorm $\gamma$ parameters under min-max quantization. The top row shows the original $\gamma$ matrices before quantization and the corresponding 12-bit min-max quantized result, where scale distortion is observed. The bottom row illustrates the same matrices after relaxation, prior to quantization: the relaxed parameters remain closer to the original, and even under a 4-bit representation exhibit improved stability compared to the unrelaxed 12-bit case. . . . . | 33 |
| 4.1 | Comparison of exponential function through different orders of polynomial approximations and log based approximation. . . . .                                                                                                                                                                                                                                                                                                                                                                                 | 36 |
| 4.2 | Block-level overview of combined first-order exponent and dot product unit. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                           | 40 |
| 4.3 | Block-level overview of compressed post exponent values. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                              | 41 |
| 4.4 | Approximation error of the proposed TR nonlinear operators (top: TR-exp, bottom: TR-ln) under 12-bit integer arithmetic. . . . .                                                                                                                                                                                                                                                                                                                                                                              | 43 |
| 4.5 | Comparison of division noise between the proposed method and a low-cost comparable solution (SOLE). The plot reports the absolute error with respect to a high-precision reference across the evaluated input range. . . . .                                                                                                                                                                                                                                                                                  | 44 |

|     |                                                                                                                                                                |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.6 | Division noise comparison across different input ranges and bit-precisions.<br>Noise is measured as absolute error with respect to a high-precision reference. | 45 |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----|

# Chapter 1

## Introduction

In recent years, transformer networks – particularly their attention mechanisms – have driven transformative advancements in artificial-intelligence (AI). These architectures have reshaped various applications, from sequence-to-sequence tasks like text summarization, machine translation, and speech recognition to vision tasks, redefined as sequence-to-sequence problems with the advent of vision transformers (ViTs). By applying this flexible architecture across diverse domains, transformers have significantly enhanced AI’s capabilities. However, despite their strengths, certain components within these models, especially the attention mechanism, pose hardware implementation challenges and therefore affect the performance.

The attention mechanism involves several non-linear operations, primarily focusing on three functions: SoftMax, Gaussian Error Linear Unit (GELU), and layer normalization (LayerNorm). From a hardware standpoint, these functions are expensive due to their use of dividers, which are multi-cycle components that are costly in terms of performance, power, and area (PPA).

The SoftMax and GELU functions incorporate exponential operations, which act as significant computational bottlenecks compared to linear arithmetic operations like multiplication and addition. Their inherent non-linearity makes approximation challenging without substantial accuracy loss, and their exponential growth often triggers numerical instability with high-magnitude inputs. Consequently, established frameworks such as FasterTransformer [1], SmoothQuant [2], AWQ [3], and GPTQ [4] focus primarily on linear layer quantization, reverting to floating-point arithmetic for non-linear operations to preserve model fidelity.

In contrast, recent efforts aim for fully integer pipelines to eliminate casting overhead. Approaches like NN-LUT [5] and IntAttention [6] utilize look-up tables (LUTs) to map these operations to hardware-efficient memory accesses. Beyond basic lookup strategies, recent research focuses on optimizing the hardware footprint by either minimizing table requirements or eliminating them entirely. For instance, Softermax [7] utilizes piecewise

linear approximation to reduce the complexity and size of fixed-point lookup tables, while integer-only techniques like I-BERT [8] and I-ViT [9] aim to eliminate LUTs altogether by substituting the exponential function with low-order polynomials or hardware-native base-2 bit-shifting operations. Subsequent studies aimed to improve efficiency by employing only simple arithmetic operations such as shifts and additions. The work in [10] was among the first to propose this approach, introducing a shift-based approximation for the exponential function and an optimized binary-logarithmic division method originally proposed by Mitchell [11], and later refined in [12] and [13].

However, even when SoftMax and GELU are converted to integer-friendly forms, LayerNorm remains a primary bottleneck. It entails both division and square-root operations, which are difficult to approximate with high fidelity and expensive to realize in fixed-point hardware. As demonstrated in [14], the LayerNorm unit alone can occupy up to 25% of the total chip area. While some works address the square and square-root computations [10], limited attention is paid to the scaling induced by the  $\gamma$  parameter, which we observe can severely degrade accuracy if improperly quantized. Moreover, most prior approaches overlook the substantial variance differences between vision and language workloads, leading to dramatically different effective dynamic ranges. QUARK [15] tackles nonlinear computation by reformulating it entirely in the log domain using second-order polynomial approximations and introduces reorder-based group quantization to alleviate inter-channel activation heterogeneity introduced by nonlinear layers such as LayerNorm and GELU.

While the approaches discussed above achieve acceptable accuracy on many benchmark tasks, they tend to come with substantial hardware cost. This is mainly due to high resource demands and intricate bit-level control logic. In addition, a number of methods still rely on high-precision arithmetic, often 32-bit operations such as division, which limits parallelism, increases area, and ultimately hurts throughput and energy efficiency. Low-precision variants can reduce some of this overhead, but they introduce different bottlenecks. For instance, they frequently depend on dedicated LUTs that must cover a wide dynamic range to support LayerNorm, division, and square-root computations. Moreover, these designs typically do not leverage existing General Matrix Multiplication (GEMM) and re-quantization kernels, which would otherwise provide efficient reuse and implementation simplicity. As a result, the overall hardware path becomes more complicated and can suffer noticeable accuracy degradation. One concrete example is treating SoftMax and sigmoid as interchangeable: despite their structural similarity, using a single SoftMax unit to implement both often signals an incomplete solution and suggests that additional refinement is needed.

In this work, we propose a unified, integer-only optimization framework for post-

---

training quantization (PTQ) of transformer nonlinear layers and demonstrate it in a fully 8-bit, integer-only network. By restructuring nonlinear computations and their associated parameters into low-precision formulations, our method eliminates the need for calibration data for nonlinearities, specialized observers, or floating-point arithmetic units.

We validate the proposed integer-only framework across both vision and language transformers. For vision tasks, even a minimal rounding-based approximation with small LUT maintains strong accuracy, and a first-order refinement further reduces the gap to full precision while preserving full INT8 quantization. For language tasks, we show that carefully reformulating normalization in low precision is essential, enabling near-FP32 performance across GLUE benchmarks where naive quantization otherwise fails. Together, these results highlight the practicality and generality of the approach for efficient transformer deployment.

Our specific contributions include the following:

- We demonstrate that accuracy degradation in post-training quantization of transformer nonlinearities is driven by a small set of structural error sources rather than by insufficient numerical precision. This finding challenges the prevailing assumption that nonlinear layers fundamentally require high-precision floating-point arithmetic.
- We introduce a unified, integer-only formulation for transformer nonlinear operations built around shared Taylor Region exponential and logarithm primitives. This enables hardware-efficient implementations of SoftMax, GELU, and Layer-Norm without floating-point fallback while exploiting the standard GEMM and re-quantization kernels.
- We identify and address two under-emphasized bottlenecks in PTQ: (i) compounded approximation effects in nonlinear activations such as GELU, and (ii) the dominance of learned scale parameters ( $\gamma$ ) in normalization layers. We propose a simple, calibration-free outlier-aware optimization to effectively mitigate these errors.
- We validate the proposed approach on vision and language transformers, demonstrating consistent performance under aggressive quantization while significantly simplifying the PTQ pipeline.





# Chapter 2

## Background

### 2.1 Transformer overview

Transformers, a type of encoder-decoder architecture, enhance sequence modeling by replacing recurrent structures with self-attention mechanisms [16]. Unlike traditional networks, encoder-decoder models can handle variable-length input-output sequences. They work by compressing the input into a context vector and generating the output using the decoder, capturing temporal dependencies that traditional architectures cannot. This architecture efficiently processes longer sequences by allowing direct attention across all positions, significantly improving the model's ability to capture both local and global context, thereby enhancing its overall understanding. The central mechanism in the transformer model is the self-attention mechanism.

#### 2.1.1 Multi-Head-Self-Attention (MHSA)

The attention mechanism is composed of several layers. It first handles an input sequence of  $m$  elements, each having  $d$  dimensions. In every step, three linear transformations are executed: Query ( $\mathbf{Q}$ ), Key ( $\mathbf{K}$ ), and Value ( $\mathbf{V}$ ). The input can be divided into  $h$  heads, leading to embedding dimensions of  $d_q = d_k = d_v = d/h$ , with each head processing part of the input's dimensionality. In the attention layer, the  $\mathbf{Q}$  and  $\mathbf{K}$  matrices undergo multiplication and are then scaled by  $\sqrt{d_k}$ . Following this, they are normalized using a SoftMax function, which applies a normalized exponential transformation. The resulting values are then multiplied by the matrix  $\mathbf{V}$  to produce the final weighted attention output:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax} \left( \frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d_k}} \right) \cdot \mathbf{V}. \quad (2.1)$$

When employing multiple attention heads in the context of  $h_i = \text{Attention}_i(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$ , the outputs of each attention head are concatenated to capture diverse representation

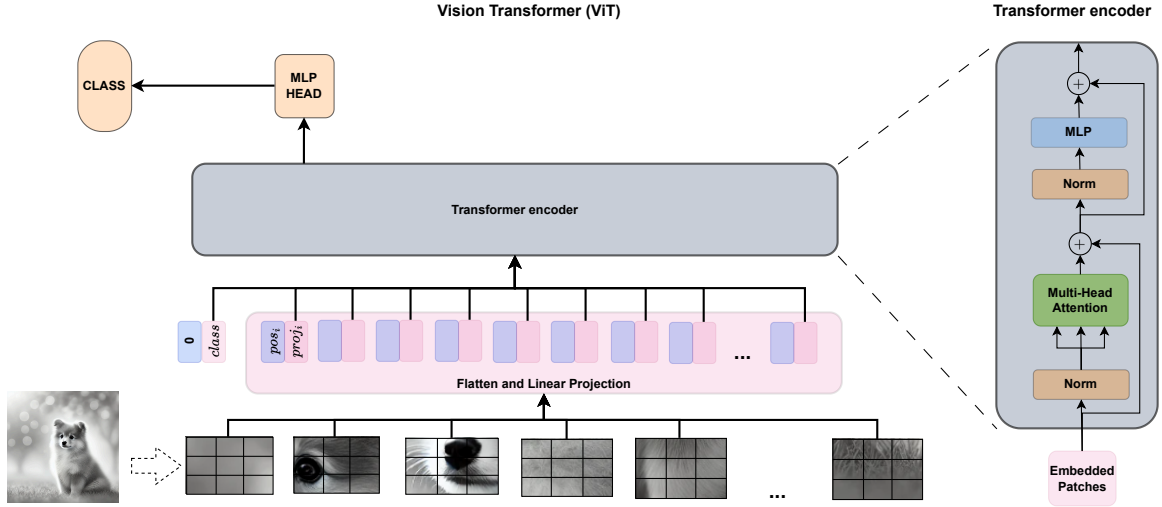


Figure 2.1: High-level illustration of the Vision Transformer.

subspaces across different positions. Subsequently, these concatenated outputs are subject to weighting by a matrix  $\mathbf{W}^o$  to yield the final output:

$$\text{MHSA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = (h_0, h_i, \dots, h_i) \cdot \mathbf{W}^o. \quad (2.2)$$

## 2.2 Vision Transformers (ViTs)

Vision transformers have significantly impacted the field of computer vision by providing an alternative to traditional convolutional neural networks (CNNs). Unlike CNNs, which rely on local convolutional filters, ViTs use self-attention mechanisms to capture long-range dependencies and global context in images. This approach allows them to excel in tasks where understanding complex spatial relationships is crucial, such as image classification, object detection, and segmentation.

The ViT, introduced by [17], is a pure transformer that directly applies to sequences of image patches for image classification tasks, closely following the original design of the transformer. Fig. 2.1 shows the framework of a ViT.

To handle 2D images, the image  $X \in \mathbb{R}^{h \times w \times c}$  is reshaped into a sequence of flattened 2D patches  $X_p \in \mathbb{R}^{n \times (p^2 \cdot c)}$ , where  $(h, w)$  is the resolution of the original image,  $c$  is the number of channels, and  $(p, p)$  is the resolution of each image patch, as shown in Fig. 2.2. The effective sequence length for the transformer is, therefore,  $n = \frac{hw}{p^2}$ .

Because the transformer uses constant widths in all layers, a trainable linear projection maps each vectorized patch to the model dimension  $d$ , and the output is referred to as *patch embeddings*. Similar to BERT's [CLS] token [18], a learnable embedding is prepended to the sequence of patch embeddings, with the state of this embedding serving as the image

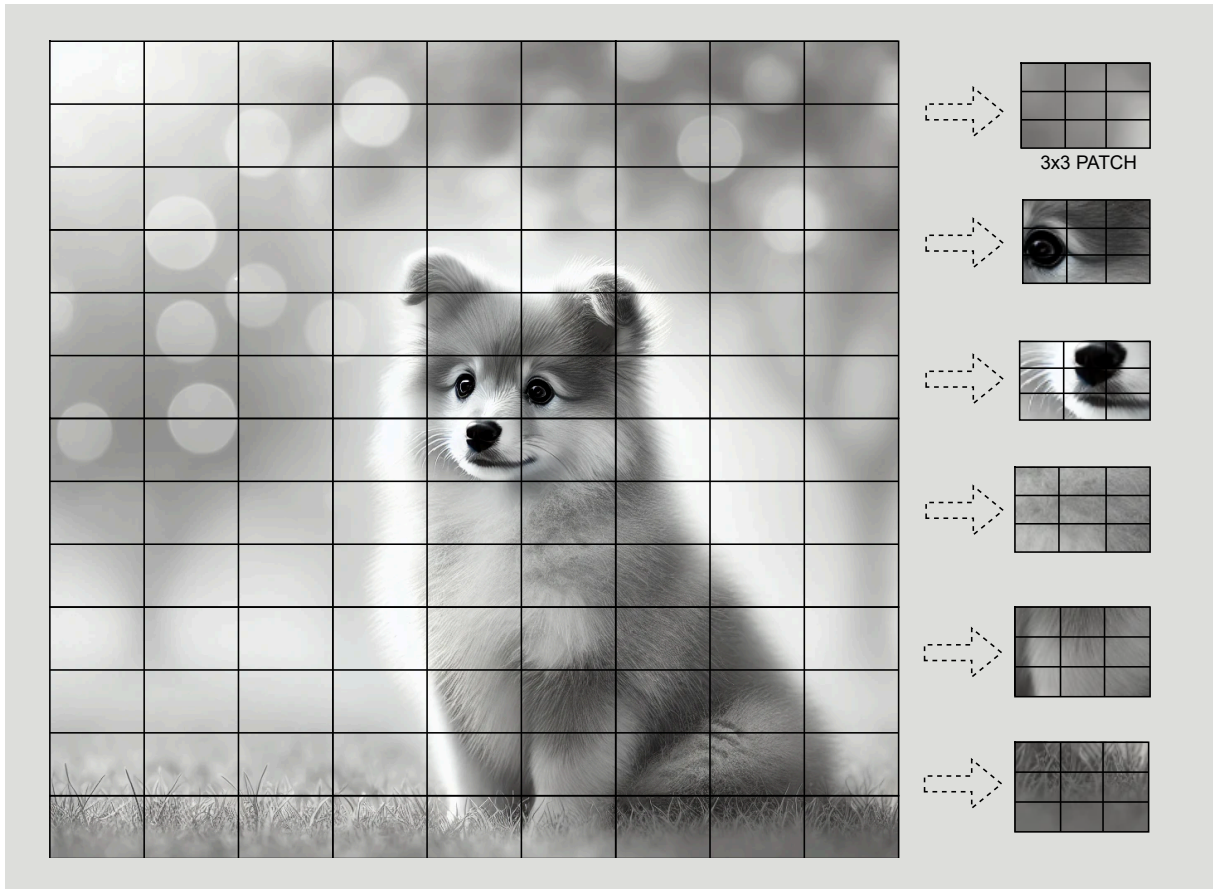


Figure 2.2: An example of an image divided into patches, each containing a  $3 \times 3$  grid of smaller cells.

representation.

The classification heads are attached at the same position during both pre-training and fine-tuning stages. Additionally, 1D position embeddings are added to the patch embeddings to retain positional information. Notably, ViTs utilize only the standard encoder of a transformer (except for a few adjustments in LayerNorm), with the encoder output followed by a multi-layer perceptron (MLP) head. ViTs are typically pre-trained on large datasets and then fine-tuned on smaller datasets for downstream tasks.

Several efforts have been made to improve the accuracy of ViTs. Designed for data efficiency, DeiT [19] achieves high performance with less data by using data augmentation and a distillation token to learn from a CNN teacher model. It retains a similar structure to ViT but requires fewer samples. Swin Transformer [20] applies self-attention within small, shifted windows, efficiently capturing local and global features. Its hierarchical structure makes it suitable for tasks like object detection and segmentation.

While transformers are capable of achieving state-of-the-art results, they present distinct challenges from a hardware perspective, particularly concerning memory require-

ments and latency due to their architecture. The memory footprint of the transformer is heavily impacted by self-attention, which requires each token to attend to all others, creating a memory complexity of  $O(n^2)$ , where  $n$  is the length of the sequence. The sequence length impacts the transformer’s performance, alongside inputs and weights. The transformer includes dense matrix multiplication (matMul) operations and requires significant hardware resources for certain layers, which diminishes computational parallelism. This, in turn, leads to increased storage of intermediate results. Accessing this memory is considerably slower and consumes more energy.

Techniques such as pruning, quantization, and memory-efficient attention (e.g., sparse attention) help reduce the footprint. This work focuses on quantization.

## 2.3 Quantization

Quantization is a technique used to reduce the precision of a neural network’s weights and activations, typically converting them from high-precision (e.g., 32-bit floating-point (FP32)) to lower precision (e.g., 8-bit integer (INT8)). This reduction improves the computational efficiency, memory usage, and power consumption of the model, making it well-suited for deployment on resource-constrained devices. There are two primary approaches to quantization: quantization-aware training (QAT) and PTQ.

In QAT, quantization is seamlessly integrated into the training process. The model is trained with simulated quantization, which allows it to learn and adjust to lower precision, minimizing accuracy loss. Although it requires additional training resources and complexity, this method achieves higher accuracy, which makes it suitable for applications where precision is critical.

PTQ applies quantization after training, with no modification to the original training process. This approach, which is simpler and faster to deploy, may introduce minor accuracy loss, as the model is not adjusted for quantization effects. PTQ is ideal for applications where speed and resource savings are prioritized over absolute highest accuracy. PTQ is a straightforward and efficient technique, particularly in resource-constrained deployments. PTQ is applied after training, requiring no model changes or retraining, which makes it easy to implement in existing workflows. It reduces memory and computational needs, leading to lower power consumption, ideal for edge and mobile devices. Furthermore, PTQ allows for rapid deployment without the time and cost of retraining. In general, high accuracy is maintained in applications with moderate precision demands, especially when combined with calibration.

### 2.3.1 Uniform Quantization

Uniform quantization reduces the precision of continuous values by mapping them to a finite set of evenly-spaced discrete levels. This approach is commonly used to quantify weights and activations in neural networks for efficient hardware deployment. The quantization levels are evenly spaced across the range of values in uniform quantization. Given a range  $[\min, \max]$  and  $n$  discrete levels, the step size  $\Delta$  is defined as:

$$\Delta = \frac{\max - \min}{n - 1}. \quad (2.3)$$

Each value  $x$  is then quantized as:

$$Q(x) = \text{round} \left( \frac{x - \min}{\Delta} \right) \cdot \Delta + \min. \quad (2.4)$$

In neural networks, uniform quantization is widely used to convert high-precision weights and activations to lower-precision formats (e.g., from FP32 to INT8). This reduces memory usage and computation time, making it suitable for deployment on resource-constrained devices. Though computationally straightforward and efficient for uniformly distributed data, uniform quantization can lead to errors if the data does not follow this distribution. This problem becomes more pronounced when quantizing non-linear layers found in transformer models, which include three significant non-linear components: SoftMax, GELU, and LayerNorm.

### 2.3.2 Re-quantization

There are two different approaches of using quantization. Scale/descale is useful in quantization schemes where hardware supports mixed precision or where accuracy is a priority. These methods apply float scaling factors at layer inputs or outputs, which preserves precision and simplifies training (e.g., in QAT or per-channel quantization). This approach often achieves better accuracy and flexibility, but at the cost of higher compute or power, making it less suitable for ultra-constrained, integer-only hardware where strict re-quantization is necessary.

Re-quantization schemes introduce a few additional operations between layers by converting intermediate results back into a quantized representation. These methods reuse the learnable scale factor from the quantization process for both the output and input of adjacent layers. Let  $R$  be the real number,  $S$  the scale factor,  $q$  the quantized integer, and  $z$  the zero-point offset. The base quantization equation is:

$$R = S \cdot (q + z). \quad (2.5)$$

Let  $R_1$  be the real-valued output of one layer and  $R_2$  the real-valued input to the next. Re-quantization sets these equal and isolates the new quantized value  $q_2$ :

$$\begin{aligned} R_2 &= R_1 \\ S_2 \cdot (q_2 + z_2) &= S_1 \cdot (q_1 + z_1). \end{aligned} \tag{2.6}$$

Solving for  $q_2$  gives:

$$q_2 = \frac{S_1}{S_2} \cdot (q_1 + z_1) - z_2. \tag{2.7}$$

Equation 2.7 introduces two challenges: representing the scale factor  $M = \frac{S_1}{S_2}$  as an integer, and ensuring that the result stays within the target bit precision.

We decompose  $M$  into a product of a power-of-two shift and a fractional multiplier:

$$M = 2^{-n} \cdot M_0 \quad \text{where} \quad M_0 \in [0.5, 1). \tag{2.8}$$

To implement this efficiently, we approximate  $M_0$  as an integer multiplier:

$$M_{s0} = \text{round}(M_0 \cdot 2^b) \tag{2.9}$$

Substituting into Equation 2.7 yields the following:

$$q_2 = \left( \frac{M_{s0} \cdot (q_1 + z_1)}{2^b} \right) \cdot 2^{-n} - z_2. \tag{2.10}$$

This form allows efficient implementation using integer multiply-shift operations. To ensure that the result fits within the desired bit width, we apply clamping:

$$q_2 = \begin{cases} \text{min\_val}, & \text{if } q_2 < \text{min\_val} \\ \text{max\_val}, & \text{if } q_2 > \text{max\_val} \\ \left( \frac{M_{s0} \cdot (q_1 + z_1)}{2^b} \right) \cdot 2^{-n} - z_2, & \text{otherwise} \end{cases} \tag{2.11}$$

A simple example of requantization is a dot product, where  $R_3 = R_1 \cdot R_2$ . In this case, the output scale  $S_3$  satisfies the following:

$$M_{s0} = \text{round} \left( \frac{S_1 \cdot S_2}{S_3} \cdot 2^b \right). \tag{2.12}$$

### 2.3.3 Integer-only Quantization

Quantization is a process that reduces the precision of a model’s weights and activations from high precision (e.g., FP32) to lower precision (e.g, FP16 or INT8). Earlier studies used floating point units of 16-bit or 8-bit precision (FP16/FP8) instead of the conventional FP32 format. This approach reduced memory demands and accelerated matrix multiplications. These formats proved especially useful for dense linear computations in transformer models, such as those involved in the self-attention layers, although they still require significant hardware resources, which are not always available in edge devices. Other works like FasterTransformer [1] quantized the linear layers, leaving non-linear operations, such as normalization and activation functions, as floating-point arithmetic.

Softmax [7] was the first to suggest a hardware-friendly SoftMax design by using base replacement. Their approach was to switch the exponential base from Euler’s number  $e$  to base-2 for low-precision SoftMax computations. Base-2 representation was only applicable with quantization aware training, as it showed major accuracy degradation on post-training quantization, while also utilizing non-linear operations such as floating point arithmetic.

I-BERT [8], I-ViT [9], and FQ-ViT [21] represent significant strides in the development of quantized transformer models that make advanced AI functionalities accessible on resource-constrained hardware. I-BERT stands as the first fully quantized network to achieve comprehensive low-precision computation by employing polynomial approximations for nonlinear operations without losing significant accuracy, positioning itself as a viable solution for edge devices with limited computational power. The solution is based on a function that minimizes the error between a function  $f$  and a second-order polynomial Denoted by:

$$\min_{a,b,c} \frac{1}{2} \left\| f(x) - x \cdot \frac{1}{2} \left[ 1 + L \left( \frac{x}{\sqrt{2}} \right) \right] \right\|_2^2, \quad \text{s.t.} \quad L(x) = a(x+b)^2 + c. \quad (2.13)$$

The author uses the above equation to approximate the exponential function and the  $erf$  function of the GELU function. The normalization square root is implemented with an iterative algorithm based on Newton’s method, and the division is based on a simple divider.

I-ViT extends this quantization strategy to vision transformers, which are widely used in image processing tasks. I-ViT showed a similar quantization scheme as I-BERT but offers base replacement techniques to optimize the exponent calculations needed in both the SoftMax and GELU functions, trading the natural base- $e$  for base-2, while mirroring the LayerNorm of I-BERT. By doing so, their work outperforms state-of-the-art results on computer vision tasks and even surpasses the baseline floating-point accuracy for some

models. Based on the input scale  $S_\Delta$  and the input  $I_\Delta$ , the exponential function of the quantized input  $x_q = S_\Delta I_\Delta$  can be expressed as:

$$\exp(S_\Delta I_\Delta) = 2^{S_\Delta \cdot \log_2 e}, \quad (2.14)$$

where can be approximated by  $\log_2 e \approx 1.4375 = 1 + 0.5 - 0.0625$ . This constant can be efficiently implemented using only shift operations:

$$\exp(S_\Delta I_\Delta) = 2^{S_\Delta \cdot (I_\Delta + (I_\Delta \gg 1) - (I_\Delta \gg 4))}. \quad (2.15)$$

To realize this using shift-only arithmetic, the result is decomposed into integer and fractional parts:

$$S_\Delta \cdot (I_\Delta + (I_\Delta \gg 1) - (I_\Delta \gg 4)) = -q + S_\Delta \cdot (-r). \quad (2.16)$$

Accordingly, the exponential function can be rewritten as

$$\exp(S_\Delta I_\Delta) = 2^{-q + S_\Delta \cdot (-r)} = 2^{S_\Delta \cdot (-r)} \gg q. \quad (2.17)$$

Finally, the fractional component is approximated by a simple linear function:

$$2^{S_\Delta \cdot (-r)} \approx \frac{\text{round}(S_\Delta \cdot (-r))}{2} + 1. \quad (2.18)$$

This work exploits the exponential structure of the sigmoid function employed in the GELU activation. In the following section, we discuss why other studies that adopted similar exponential formulations for the SoftMax function were unable to extend the same approach effectively to GELU, despite utilizing comparable exponential implementations. Apart from this distinction, the remainder of the architecture and methodology closely follow the design principles introduced by I-BERT.

Similar to the approach in [7], both I-BERT and I-ViT rely on quantization-aware training, a significantly more demanding process that is less suitable for hardware deployment, where retraining or fine-tuning is often impractical and computationally expensive. In contrast, FQ-ViT adopts a post-training quantization-based methodology, enabling post-training quantization without the need for backpropagation or dataset-dependent retraining. FQ-ViT reuses the SoftMax implementation from I-BERT, leveraging the approximately normal distribution of SoftMax outputs to perform an aggressive 4-bit quantization. This is achieved through a  $\log_2$ -domain quantization strategy followed by



the transformation expressed as:

$$\text{Attn}_Q = Q(\text{Attn} \mid b) = \text{clip}(\lfloor -\log_2(\text{Attn}) \rfloor, 0, 2^b - 1). \quad (2.19)$$

Following this transformation, the subsequent multiplication by the value matrix can be efficiently replaced with a simple shift operation, thereby eliminating the need for costly floating-point or integer multiplications and further reducing the computational complexity of the attention module.

$$\text{Attn}_Q \cdot V^T = 2^{(-\text{Attn}_Q)} \cdot V^T = V^T \gg \text{Attn}_Q. \quad (2.20)$$

In addition the author mentioned the variations that characterize the LayerNorm inputs and suggests a method to handle it, using *Power-of-Two Factor* (PTF), as shown in Fig. 2.3 for example. It uses the standard quantization with:

$$\mathbf{X}_Q = Q(\mathbf{X} \mid b) = \text{clip}\left(\left\lfloor \frac{\mathbf{X}}{2^{\alpha_s}} \right\rfloor + zp, 0, 2^b - 1\right), \quad (2.21)$$

$$s = \frac{\max(\mathbf{X}) - \min(\mathbf{X})}{2^b - 1} \cdot 2^{-K}, \quad (2.22)$$

$$zp = \text{clip}\left(\left\lfloor -\frac{\min(\mathbf{X})}{2^{K_s}} \right\rfloor, 0, 2^b - 1\right), \quad (2.23)$$

$$\alpha_c = \arg \min_{\alpha \in \{0, 1, \dots, K\}} \left\| \mathbf{X}_c - \left( \left\lfloor \frac{\mathbf{X}_c}{2^{\alpha_s}} \right\rfloor \right) 2^{\alpha_s} \right\|_2, \quad (2.24)$$

when the goal is to find the best power of two factor  $2^{\alpha_c}$  to minimize the LayerNorm inter-channel variation and  $k$  is a hyperparameter that in practice equals  $k = 3$  for the chosen models.

It is worth mentioning that even though they do not implement the GELU layer and leave it as floating point arithmetic, [22] has already suggested a range-constrained quantization technique for activation and normalization operators that takes FQ-ViT and implements it in the GELU operation but still loses some accuracy. The reasons for this are discussed in Chapter 3.

Although the above works help to approximate the non-linear layers, they all use heavy computational units such as dividers, square root, or high bit precision. In contrast, SOLE [10] aims to find a solution without even a multiplier. To do so, they suggest E2Softmax which uses Approximate Log-based Division (ALDivision), to replace the SoftMax exponent and division with shift, addition and mux operations. This method approximates  $e_Q^X$  similar to I-ViT using 2.15 and computes the division in the log domain:

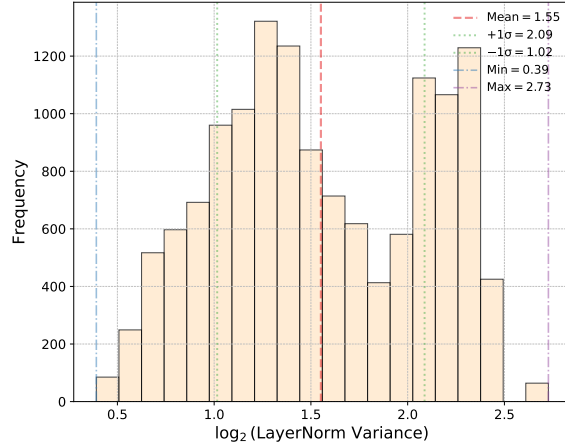


Figure 2.3: Statistical distribution of variance across layers in the `bert-uncased-whole-word-masking` model evaluated on the SQuAD dataset.

$$Q_Y = \log_2(\exp X_Q) = S_\Delta \cdot (I_\Delta + (I_\Delta \gg 1) - (I_\Delta \gg 4)). \quad (2.25)$$

This solution aggressively approximates the divider by decomposing the denominator to a power of two and a number in the group 0,0.5:

$$\frac{Q_y}{S} = 2^{-(k_y+k_s)} \cdot \frac{1}{1+s}, \quad (2.26)$$

$$q(s) = \left\lfloor \frac{2 \cdot s}{2} \right\rfloor, \quad q(s) \in \{0, 0.5\}, \quad (2.27)$$

$$\frac{Q_y}{S} \approx 2^{-(k_y+k_s+1)} \cdot (1 - q(s)), \quad (2.28)$$

$$\text{ALDivision}(k_y, S) = 2^{-(k_y+k_s+1)} \cdot (1.636 - q(s)). \quad (2.29)$$

The result has the same 4-bit structure as 2.20.

QUARK [15] achieves divider-free nonlinear evaluation by reformulating division in the log domain and reusing shared sub-operators across SoftMax and LayerNorm. Specifically, QUARK rewrites SoftMax via a log-sum-exp (LSE) form:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} = \exp\left(x_i - \ln\left[\sum_{j=1}^n \exp(x_j)\right]\right). \quad (2.30)$$

Similar to I-ViT, QUARK replaces base- $e$  exponentiation with a base-2 formulation and decomposes the exponent into integer and fractional components to enable shift-based

realization:

$$\exp(x'_i) = 2^{x'_i \log_2 e} \approx 2^{x'_i + (x'_i \gg 1) - (x'_i \gg 4)} = 2^{q_I + q_F} = 2^{q_I} \cdot 2^{q_F}, \quad (2.31)$$

$$q_I = \lceil x'_i + (x'_i \gg 1) - (x'_i \gg 4) \rceil, \quad q_F = x'_i + (x'_i \gg 1) - (x'_i \gg 4) - q_I, \quad (2.32)$$

$$q_F \in (-1, 0], \quad 2^{q_I} \cdot 2^{q_F} = (2^{q_F}) \ll q_I, \quad (2.33)$$

$$2^{q_F} \approx 0.1713 q_F^2 + 0.6674 q_F + 0.998. \quad (2.34)$$

To complete the log-domain pipeline, QUARK also approximates the natural logarithm by first converting it to a base-2 logarithm and then implementing the constant factor  $\ln 2$  using shifts and additions:

$$\ln(x) = (\ln 2) \log_2(x), \quad \ln 2 \approx (0.1011)_2, \quad (2.35)$$

$$\ln(x) \approx \log_2(x) - (\log_2(x) \gg 2) - (\log_2(x) \gg 4). \quad (2.36)$$

where  $\log_2(x)$  is decomposed by expressing  $x$  as a power-of-two times a normalized mantissa:

$$x = 2^{q_M} q_N, \quad q_N \in [1, 2), \quad (2.37)$$

$$\log_2(x) = \log_2(2^{q_M} q_N) = q_M + \log_2(q_N), \quad (2.38)$$

$$\log_2(q_N) \approx -0.3369 q_N^2 + 1.995 q_N - 1.65. \quad (2.39)$$

Beyond SoftMax, QUARK applies the same log-domain division elimination to Layer-Norm by reformulating the iterative square-root computation. In particular, the Newton-style update uses a division term  $\text{Var}(x)/x_i$ , which is replaced by exponentiating the difference of logarithms:

$$x_{i+1} = \frac{x_i + \left\lfloor \frac{\text{Var}(x)}{x_i} \right\rfloor}{2} = \left( x_i + \left\lfloor \frac{\text{Var}(x)}{x_i} \right\rfloor \right) \gg 1, \quad (2.40)$$

$$\frac{\text{Var}(x)}{x_i} = \exp\left(\ln\left[\frac{\text{Var}(x)}{x_i}\right]\right) = \exp(\ln(\text{Var}(x)) - \ln(x_i)), \quad (2.41)$$

$$x_0 = 2^{\lfloor \text{bit}(\text{Var}(x))/2 \rfloor}. \quad (2.42)$$

As a result, QUARK avoids dedicated divider hardware by reusing the same integer-only  $\exp(\cdot)$  and  $\ln(\cdot)$  approximations across multiple nonlinear operators.

### 2.3.4 Limitations of prior works

While prior work has significantly advanced integer-only or low-precision transformer inference, existing approaches remain architecturally fragmented and rely on layer-specific approximations that preclude the design of a unified shared hardware unit for nonlinear operations.

I-BERT [8] preserves 32-bit arithmetic in critical paths and employs second-order interpolation to approximate nonlinear functions. Although numerically stable, this approach is not well aligned with GEMM-centric accelerator architectures, as it introduces high-precision datapaths and interpolation logic that cannot be efficiently fused with standard integer matrix-multiplication pipelines.

I-ViT [9] improves computational efficiency relative to I-BERT by reducing precision and optimizing certain nonlinear kernels. However, it continues to rely on explicit divider units and multi-cycle iterative approximations, thereby retaining long-latency control-dependent operations that hinder full parallelization.

FQ-ViT [21] further reduces quantization error through fine-grained scaling strategies, yet it still incorporates hardware dividers and multi-cycle square-root implementations similar to those in I-BERT and I-ViT. Moreover, GELU is not fully realized in integer arithmetic, limiting end-to-end hardware homogeneity.

SOLE [10] replaces nonlinear operators with shift-based approximations that exploit the empirically observed insensitivity of SoftMax to small perturbations. However, this assumption does not generalize to architectures with frequent or stacked SoftMax layers, where accumulated error may become significant. In addition, the method relies on large lookup tables for inverse square-root computation, increasing memory footprint and limiting scalability.

Quark [15] eliminates explicit division through second-order polynomial and iterative formulations. While this removes dedicated divider hardware, it substantially increases multiplication count, thereby trading control-flow complexity for arithmetic intensity and sustained multiplier utilization.

Collectively, these methods share a common limitation: nonlinear operations are addressed using heterogeneous, operator-specific mechanisms: interpolation units, dividers, iterative solvers, shift heuristics, or large lookup tables, rather than through a unified computational primitive. This fragmentation prevents the construction of a single shared nonlinear processing unit that can be reused across SoftMax, normalization, GELU, and inverse square-root operations. Consequently, hardware implementations must either instantiate multiple specialized units or tolerate multi-cycle serialized execution, both of which inflate area, increase latency, and reduce throughput.

Therefore, despite substantial progress in quantized transformer deployment, division-

class operators and related nonlinear functions remain fundamental architectural bottlenecks, characterized by significant area overhead, limited parallel scalability, and incompatibility with streamlined integer-only datapaths.



## Chapter 3

# Efficient Nonlinear Computation for Quantized Transformer Models

### 3.1 Integer-only Taylor Region Approximation for Non-linear Functions

In this section, we present unified, integer-only approximations for the exponential ( $\exp$ ) and natural logarithm ( $\ln$ ) functions using a Taylor-based integer representation. These nonlinear functions serve as central components in essential deep-learning operations, such as SoftMax and GELU. Consequently, these approximations form the fundamental building blocks for the complete operators described in the subsequent section.

The proposed Taylor Region  $\exp$  (TR- $\exp$ ) and  $\ln$  (TR- $\ln$ ) approximations operate **entirely in the integer domain** using shift and multiply operations. This design is highly efficient for quantized hardware accelerators and low-power processing units. Both functions are expressed in terms of a base evaluation point, or *anchor*  $a_i$  (derived from the leading bits of the input), and a small offset calculated based on the residual distance ( $\Delta$ ) of the input from the evaluation point. The evaluation leverages truncated Taylor series and power-of-two shifts instead of complex division logic. This shared structure simplifies the reuse of arithmetic logic and lookup tables across both nonlinearities. Following the introduction of the methods for calculating  $\exp$  and  $\ln$ , we will demonstrate the full integer-only computations for SoftMax, GELU, and LayerNorm.

#### 3.1.1 Taylor Region-Based Approximation of the Exponential Function (TR- $\exp$ )

The proposed approximation of the exponential function (TR- $\exp$ ) relies on bounding the input domain, quantizing the values, and utilizing a small LUT with pre-calculated

evaluation points to pin a low-order Taylor series approximation.

The first step requires bounding the inputs to ensure they are non-positive. This is achieved by subtracting the maximum value  $x_{\max}$  from the original inputs  $x$ , yielding  $\hat{x} = x - x_{\max} \leq 0$ . This effectively bounds the output within the range  $\exp(\hat{x}) \in (0, 1]$ .

To address quantization, we interpret the binary layout of the quantized input values as three distinct fields: a 1-bit *sign*, an  $l$ -bit *evaluation point* (*evp*), and a  $k$ -bit *fraction* (*frac*). For a  $b$ -bit quantization scheme where  $b = l + k$ , the structure is defined as follows<sup>1</sup>:

$$\underbrace{\left[ \underbrace{\text{sign}}_{1\text{-bit}} \mid \underbrace{\text{evp}}_{l\text{-bits}} \mid \underbrace{\text{frac}}_{k\text{-bits}} \right]}_{b\text{-bits}}. \quad (3.1)$$

We then quantize the inputs to  $2^b$  discrete levels by scaling by  $2^k$  while clamping the lower bound. Since the exponent of inputs with very large negative magnitudes approaches zero, we define:

$$\hat{x}_q = \begin{cases} \left\lfloor \hat{x} \cdot 2^k \right\rfloor & \text{if } \hat{x} > -2^b, \\ -2^b & \text{if } \hat{x} \leq -2^b. \end{cases} \quad (3.2)$$

Next, we use the *evp* field to extract evaluation points for the exponent calculation. This is performed by pre-calculating  $2^l$  exponents  $\exp(a_i)$ , where  $a_i \in \{0, \dots, 2^l\}$  is the anchor. These values are stored in a small LUT and represent the nearest integer to which the Taylor series is pinned. The proposed format allocates  $l + 1$  bits: a single sign bit and  $l$  index bits that address the LUT. The motivation for this approach is illustrated in Fig. 3.1, which plots the  $\exp$  function with the Taylor approximation around the anchor points.

All tabulated values correspond to strictly negative exponents; therefore, the LUT contains exactly  $2^l$  entries. The value associated with zero input,  $\exp(0) = 1$ , is handled as a designated non-LUT case encoded directly by the sign bit. This eliminates the need to reserve a LUT location for the zero exponent while preserving a uniform indexing structure for all negative inputs.

Recall that the  $n$ -th order Taylor approximation of the exponential function around an anchor point  $a$  is given by:

$$\exp(x) \approx e^a \sum_{i=0}^n \frac{(x - a)^i}{i!}, \quad (3.3)$$

where  $x$  is the input and  $a$  denotes the chosen evaluation point. For  $n = 2$ , the exponential

---

<sup>1</sup>Since all inputs are non-positive, the sign value is implicit, meaning only  $b$  bits are strictly required for storage.



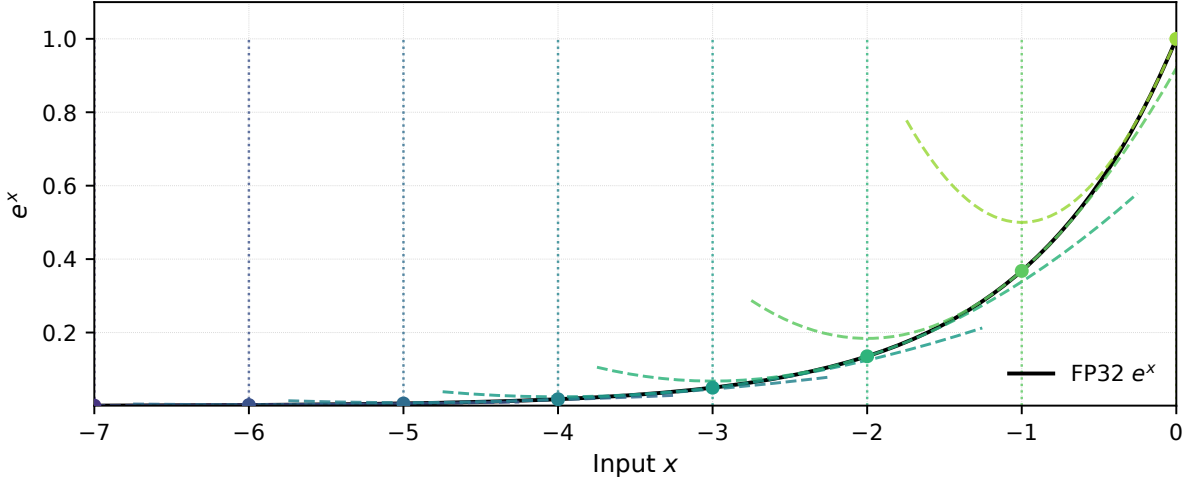


Figure 3.1: Taylor Regions for  $e^x$ . Vertical dotted lines denote anchor points; dashed curves show local Taylor approximations centered at each anchor.

of a quantized and range-limited input  $\hat{x}_q$  can be approximated as:

$$\exp(\hat{x}_q) \approx \text{LUT}[\text{evp}] \left( 1 + \Delta + \frac{\Delta^2}{2} \right), \quad (3.4)$$

where  $\text{LUT}[\text{evp}]$  stores the precomputed anchor value corresponding to the index  $\text{evp}$ , and  $\Delta = \hat{x}_q - (\text{evp} \ll k)$  represents the local offset from the anchor. However, all factors in (3.4) must be scaled to  $b$ -bit precision using the scaling factor  $2^k$ . The corrected integer-only expression is:

$$\exp(\hat{x}_q) \approx \text{LUT}[\text{evp}] (1 \ll k + \Delta + (\Delta^2 \gg (k+1))) \gg k. \quad (3.5)$$

Here, the quadratic term is rescaled by right-shifting by  $k$ , and the division-by-two is achieved with an additional bit-shift (i.e.,  $\gg (k+1)$ ). The result is rescaled by right-shifting the final product by  $k$ .

An additional benefit of this approach is that  $\Delta$  only requires  $k$  bits, since the  $l$  MSBs of  $\hat{x}_q$  are equal in each segment  $\text{evp}$  and therefore cancel out during subtraction. Consequently, the calculation of  $\Delta$  demands low bit precision and can be calculated once, then efficiently pipelined for the calculation of each additional order of the Taylor series. This structure also allows precision to be tuned based on accuracy requirements, as shown in Section 4.

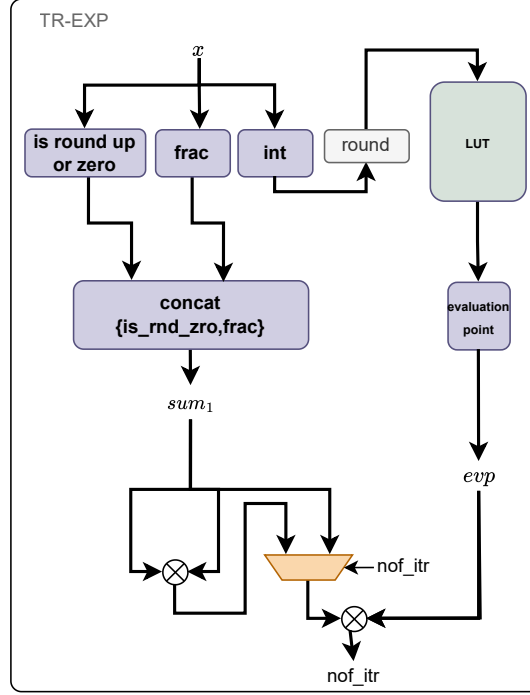


Figure 3.2: Block-level overview of TR-exp.

### Hardware-Efficient Taylor Implementation

The primary motivation of the Taylor-Region (TR) formulation is to reduce transcendental functions to operations that map efficiently onto fixed-point hardware. By aligning the  $Q_{I,K}$  representation with the Taylor expansion centers, the proposed design avoids high-latency arithmetic units and enables compact, deterministic implementations.

**Rounding-based Taylor centering.** To minimize approximation error, we center the Taylor expansion at the nearest integer value  $a = \text{round}(x)$  rather than at  $a = \lfloor x \rfloor$ . In a  $Q_{I,K}$  fixed-point representation, this rounding is implemented by inspecting the most significant fractional bit  $x[k-1]$ . Specifically,

$$a = \begin{cases} \lfloor x \rfloor + 1, & \text{if } x[k-1] = 1, \\ \lfloor x \rfloor, & \text{otherwise.} \end{cases} \quad (3.6)$$

This choice ensures that the residual  $\Delta x = x - a$  lies within  $[-0.5, 0.5]$ , reducing the maximum approximation error by a factor of two relative to floor-based centering.

---

**Algorithm 1** TR-exp: Integer-only Taylor-Region exponential (first/second order)

---

**Require:** Fixed-point input  $\hat{x}_q \in \text{QI}.K$  with  $k$  fractional bits

**Require:**  $\text{LUT}[\cdot]$ : anchor values at evaluation points (segments)

**Require:**  $\text{order} \in \{1, 2\}$  (Taylor order)

**Ensure:** Approximation of  $\exp(\hat{x}_q)$  in fixed-point

**Bit fields:**

- 1:  $\text{frac} \leftarrow \hat{x}_q[k-1:0]$   $\triangleright$  fractional field ( $k$  bits)
- 2:  $\text{is\_rnd\_up} \leftarrow \hat{x}_q[k-1]$   $\triangleright$  MSB of fraction selects rounding
- 3:  $\text{is\_rnd\_zro} \leftarrow \neg \text{is\_rnd\_up}$   $\triangleright$  used for adder-free linear term
- 4:  $\text{evp} \leftarrow \text{segment\_id}(\hat{x}_q)$   $\triangleright$  MSBs selecting the Taylor region / anchor

**Anchor evaluation point:**

- 5:  $E_a \leftarrow \text{LUT}[\text{evp}]$   $\triangleright E_a \approx \exp(a)$  at region center

**Rounding-based centering and residual:**

- 6:  $a \leftarrow \text{round}(\hat{x}_q)$   $\triangleright$  implemented by  $\text{is\_rnd\_up}$  (Eq. 3.6)
- 7:  $\Delta \leftarrow \hat{x}_q - a$   $\triangleright \Delta \in [-0.5, 0.5]$ ; MSBs cancel, so  $\Delta$  needs only  $k$  bits

**Adder-free first-order term (Eq. 3.7):**

- 8:  $\text{lin} \leftarrow \{\text{is\_rnd\_zro}, \text{frac}\}$   $\triangleright \text{lin} \equiv 1 + \Delta$  in Q format, without a full-width adder
- 9: **if**  $\text{order} = 1$  **then**
- 10:      $T \leftarrow \text{lin}$
- 11: **else**

**Second-order correction:**

- 12:      $q \leftarrow \Delta \cdot \Delta$   $\triangleright$  can be a small multiplier *or* logic-minimized Boolean block
- 13:      $\text{quad} \leftarrow q \gg (k+1)$   $\triangleright$  implements  $\Delta^2/2$  via right shift by  $k$  and additional 1 (division-by-two)
- 14:      $T \leftarrow \text{lin} + \text{quad}$   $\triangleright t \approx 1 + \Delta + \Delta^2/2$
- 15: **end if**

**Final scaling and product (matches Eq. 3.5 structure):**

- 16:  $y \leftarrow (E_a \cdot T) \gg k$   $\triangleright$  rescale back by  $k$  fractional bits
  - 17: **return**  $y$
-

**Addition-free first-order evaluation.** The first-order Taylor term  $(1 + x - a)$  is computed without a full-width adder by exploiting the structure of the fractional field  $f = x[k - 1 : 0]$ . Depending on the rounding outcome, the expression simplifies to:

$$1 + x - a = \begin{cases} \{1, f\}, & \text{if } a = \lfloor x \rfloor, \\ \{0, f\}, & \text{if } a = \lfloor x \rfloor + 1. \end{cases} \quad (3.7)$$

In hardware, this replaces a  $b$ -bit addition with a single multiplexer controlled by  $x[=k - 1]$ , significantly reducing area and power consumption. Fig. 3.2 illustrates the block-level hardware realization of the proposed TR-exp module, supporting both first- and second-order approximations.

**Logic-based quadratic approximation.** Under rounding-based centering, the residual satisfies  $|\Delta x| \leq 0.5$ , and the quadratic term  $\Delta x^2/2$  is bounded within  $[0, 0.125]$ . For a  $Q_{4.4}$  configuration, a 4-bit signed residual maps to a 2-bit output. Exploiting this narrow range, the second-order term is implemented using combinational logic synthesized via Boolean minimization, rather than a general-purpose multiplier.

By realizing the quadratic correction as a fixed 4-to-2 bit Boolean function, the evaluation can be performed in parallel with the first-order term. As a result, the exponential  $\exp(x)$  is reduced to: (i) a small lookup table (e.g., 16 entries) for the anchor  $e^a$ , (ii) a multiplexer for the first-order linear term, and (iii) combinational logic for the second-order curvature. This structure enables single-cycle evaluation of nonlinear operators such as SoftMax and GELU, while maintaining a low hardware footprint. Algorithm 1 provides a formal description of the proposed TR-exp computation.

### 3.1.2 Shift-based Taylor Region Natural Logarithm Approximation (TR-ln)

The previous subsection introduced our Taylor Region-based approximation for the exponential function (TR-exp), a core operator in deep-learning workloads such as SoftMax. Building on this, we observe that by employing a lightweight implementation of the natural logarithm (the complementary function of the exponential), we can perform additional nonlinear operations directly in the log-domain with negligible accuracy degradation. This advantage becomes particularly pronounced when the exp operator requires higher precision. In this subsection, we present a low-cost logarithm approximation suitable for integration into computations such as SoftMax and LayerNorm.

Exploiting the sub-linear behavior of the natural logarithm over a bounded interval, we replace the full  $\ln(\cdot)$  computation with a first-order (linear) approximation. To further

reduce approximation error, the input domain is divided into piecewise-linear segments  $[a, b]$ . The segment boundaries can be selected efficiently in hardware by using the position of the leading 1-bit of  $x$  together with the position of the next most significant bit. These bits determine the interval  $2^{\lfloor \log_2 x \rfloor}$  in which the value resides, effectively selecting the appropriate linear region. As the segments widen, the function slope decreases, thereby ensuring numerical stability.

Under this construction, the logarithm of the quantized input  $x_q$  is approximated as:

$$\ln x_q = m(x_q - x_{q0}) + \ln 2^a, \quad (3.8)$$

where  $a$  denotes the position of the leading-one bit,  $x_{q0}$  is the corresponding power-of-two anchor point determined by that leading bit, and the slope  $m$  is selected according to the chosen segment. The slope can be expressed as:

$$m = \frac{\ln 2^{a+1} - \ln 2^a}{2^{a+1} - 2^a} = \frac{\ln 2}{2^a}. \quad (3.9)$$

Under these definitions and substitutions, (3.8) becomes:

$$\begin{aligned} \ln x_q &= 2^{-a} \cdot \ln 2(x - 2^a) + a \ln 2 \\ &= \ln 2(2^{-a}x - 1) + a \ln 2 \\ &= \ln 2(2^{-a}x + a - 1). \end{aligned} \quad (3.10)$$

Note that  $\ln 2$  can be approximated numerically as  $\ln 2 \approx 0.5 + 0.125 + 0.0625$ . Since these coefficients are powers of two, the associated multiplications can be realized using shift-and-add operations rather than full multipliers. Thus, defining  $k = 2^{-a}x + (a - 1)$ , we obtain:

$$k \cdot \ln 2 \approx (0.5 + 0.125 + 0.0625) k, \quad (3.11)$$

and therefore:

$$\ln x_q \approx (k \gg 1) + (k \gg 3) + (k \gg 4). \quad (3.12)$$

The resulting TR- $\ln$  computation flow is illustrated in Fig. 3.3, while Algorithm 2 summarizes the corresponding hardware-friendly implementation.

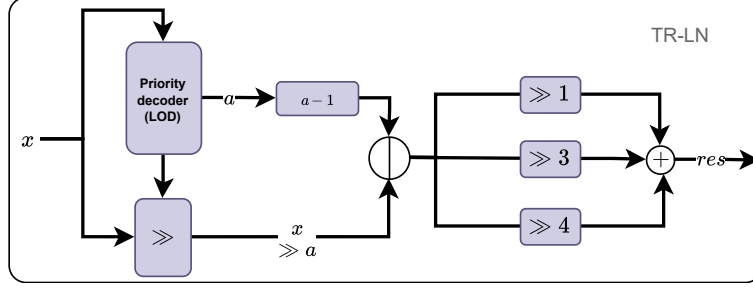


Figure 3.3: Block-level overview of TR-ln.

---

**Algorithm 2** TR-ln: Integer-only Taylor-Region logarithm approximation
 

---

**Require:** Quantized input  $x_q > 0$  in fixed-point format with  $k$  fractional bits

**Ensure:**  $y \approx \ln(x_q)$  in fixed-point

- 1:  $a \leftarrow \text{LOD}(x_q) - k$   $\triangleright$  position of leading 1-bit ( $a = \lfloor \log_2 x_q \rfloor$ )
  - 2:  $x_{\text{norm}} \leftarrow x_q \gg a$   $\triangleright$  compute  $2^{-a}x_q$  via right shift
  - 3:  $S_{ln} \leftarrow x_{\text{norm}} + (a - 1) \ll k$   $\triangleright$  from Eq. (3.10):  $S_{ln} = 2^{-a}x + (a - 1)$
  - Shift-and-add approximation of  $S_{ln} \cdot \ln 2$**
  - 4:  $y \leftarrow (S_{ln} \gg 1) + (S_{ln} \gg 3) + (S_{ln} \gg 4)$   $\triangleright$  since  $\ln 2 \approx 0.5 + 0.125 + 0.0625$
  - 5: **return**  $y$
- 

## 3.2 Hardware-Efficient TR Calculation of Nonlinear Deep Learning Operators

In this section, we introduce a unified, hardware-efficient methodology for approximating nonlinear functions based on the Taylor Region approach developed in Section 3.1. We first demonstrate the approach on SoftMax and then extend it to GELU and LayerNorm, enabling a fully quantized inference pipeline for vision and language tasks. These proposed computations reduce the cost of operations such as division, square root, and exponentiation into simple additions, shifts, and integer multiplications. This dramatically improves efficiency on resource-constrained hardware by removing the need for floating-point arithmetic units, dividers, and other complex hardware blocks.

### 3.2.1 Taylor Region SoftMax Approximation (TR-SoftMax)

Algorithm 3 describes the implementation of the SoftMax operation using the TR-exp and TR-ln primitives. Given an input vector  $x = [x_1, x_2, \dots, x_n]$  of length  $n$ , the SoftMax operator maps each element to a probability  $p_i \in [0, 1]$  such that  $\sum_{i=1}^n p_i = 1$ . Formally:

$$\text{SoftMax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)}. \quad (3.13)$$

To ensure that the exponentials do not overflow in a quantized representation, the first step is to subtract the maximum input value from all inputs,  $\hat{x} = x - x_{\max}$ . This simple modification guarantees numerical stability while leaving the output distribution unchanged. Following bounding and scaling, (3.13) can be rewritten as:

$$\text{SoftMax}(\hat{x}_{q,i}) = \frac{\exp(\hat{x}_{q,i})}{\sum_{j=1}^n \exp(\hat{x}_{q,j})}, \quad (3.14)$$

where  $\hat{x}_{q,i}$  are the quantized and bounded input values.

To compute the denominator  $S \triangleq \sum_{j=1}^n \exp(\hat{x}_{q,j})$ , we exploit the fact that at least one input satisfies  $x_j = x_{\max}$ , implying  $\hat{x}_{q,j} = 0$  and  $\exp(\hat{x}_{q,j}) = 1$ . Consequently,  $S \geq 1$  and  $\ln S \geq 0$ . This allows us to evaluate the reciprocal  $S^{-1}$  using the TR-ln and TR-exp operators:

$$S^{-1} = \exp(-\ln S), \quad (3.15)$$

where the exponent  $-\ln S$  is non-negative, as required by the TR-exp approximation.

To obtain the final SoftMax value, we multiply  $S^{-1}$  by  $\exp(\hat{x}_{q,i})$  and apply a right shift by  $k$  bits for rescaling. The complete TR-SoftMax computation is therefore:

$$\text{TR-SoftMax}[\hat{x}_{q,i}] = s_f \cdot \text{TR-exp}[\hat{x}_{q,i}] \gg k, \quad (3.16)$$

where the normalization factor  $s_f$  is computed once per vector:

$$s_f = \text{TR-exp} \left( -\text{TR-ln} \left[ \sum_{j=1}^n \text{TR-exp}(\hat{x}_{q,j}) \right] \right). \quad (3.17)$$

All operations involving additions, shifts, multiplications, and LUT lookups are implemented at  $b$ -bit precision. This enables an efficient, fully integer arithmetic realization.

### 3.2.2 Taylor Region GELU Approximation (TR-GELU)

Another primary nonlinear function used in various deep learning models is the GELU [23]. GELU combines linear and nonlinear behaviors smoothly, often yielding better convergence than ReLU in both NLP and vision models. It is defined as:

$$\text{GELU}(x) = x \Phi(x) = x \frac{1}{2} \left( 1 + \text{erf} \left( \frac{x}{\sqrt{2}} \right) \right), \quad (3.18)$$

where  $\Phi(x)$  is the Gaussian cumulative distribution function (CDF) and  $\text{erf}(\cdot)$  is the error function.

Direct evaluation of  $\text{erf}(\cdot)$  is prohibitively expensive in dedicated hardware. A common

**Algorithm 3** Taylor-Region SoftMax (TR-SoftMax)

---

**Require:** Quantized input vector  $x_q = [x_{q,1}, x_{q,2}, \dots, x_{q,n}]$ , right-shift factor  $k$ **Ensure:** Output vector  $y = [y_1, y_2, \dots, y_n]$ 

```
1:  $x_{\max} \leftarrow \max(x_q)$ 
2: for  $i = 1$  to  $n$  do
3:    $\hat{x}_{q,i} \leftarrow x_{q,i} - x_{\max}$ 
4: end for
5:  $S \leftarrow 0$ 
6: for  $i = 1$  to  $n$  do
7:    $e_i \leftarrow \text{TR-exp}(\hat{x}_{q,i})$ 
8:    $S \leftarrow S + e_i$ 
9: end for
10:  $s_f \leftarrow \text{TR-exp}(-\text{TR-ln}(S))$ 
11: for  $i = 1$  to  $n$  do
12:    $y_i \leftarrow (s_f \cdot e_i) \gg k$ 
13: end for
14: return  $y$ 
```

---

workaround replaces  $\Phi(x)$  with a scaled sigmoid:

$$\text{GELU}(x) \approx x \sigma(\alpha x), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (3.19)$$

where  $\alpha \approx 1.702$  is chosen to best fit the exact GELU curve in a least-squares sense [23]. This reduces complexity to a single multiply and a lookup-friendly logistic approximation.

An alternative exploits the “SoftMax-style” identity [9]:

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^0}{e^0 + e^{-x}} = \frac{e^{-x_{\max}}}{e^{-x_{\max}} + e^{x-x_{\max}}}, \quad (3.20)$$

where  $x_{\max} = \max(0, x)$  ensures that the exponentials are applied to non-positive numbers for numerical stability.

The GELU approximation in (3.19) and (3.20) allows the use of the same TR-exp and TR-ln operators employed earlier for TR-SoftMax. However, this sigmoid-based form is only reliable when the model was trained with the same approximation. For production models trained with the exact GELU in (3.18), substituting (3.19) at inference introduces a noticeable accuracy degradation.

To address this limitation, we replace the global scaling factor with a region-dependent parameterization. The scaling factors  $\{\alpha_g\}$  are computed offline by minimizing the mean-square error between the Sigmoid approximation and the exact Gaussian CDF over each segment. In our implementation, we use ten uniformly spaced segments spanning  $[-5, 5]$ .



For segment  $g$ , we solve:

$$\alpha_g = \arg \min_{\alpha \in \mathbb{R}} \mathbb{E}_{x \sim \mathcal{U}(\mathcal{I}_g)} \left[ (\Phi(x) - \sigma(\alpha x))^2 \right], \quad (3.21)$$

where  $\mathcal{I}_g$  denotes the  $g$ -th interval,  $\Phi(\cdot)$  is the Gaussian CDF, and  $\sigma(\cdot)$  is the Sigmoid function.

Due to the odd symmetry of GELU, i.e.,  $GELU(-x) = -GELU(x)$ , the optimal scaling factors for negative inputs are obtained by mirroring the positive regions. Consequently, only half of the coefficients need to be stored in practice, reducing the LUT to four entries without loss of accuracy. The resulting approximation procedure is summarized in Algorithm 4.

---

**Algorithm 4** Segment-Wise GELU Approximation in the Log Domain

---

**Require:** Input  $x$ , segment intervals  $\{\mathcal{I}_g\}$ , precomputed coefficients  $\{\alpha_g\}$

**Ensure:** Approximation  $\widehat{GELU}(x)$

- 1:  $u \leftarrow |x|$
  - 2: Find segment index  $g$  such that  $u \in \mathcal{I}_g$
  - 3:  $\alpha \leftarrow \alpha_g$  ▷ Select region-wise scaling factor
  - 4:  $z \leftarrow \alpha x$
  - 5:  $x_{\max} \leftarrow \max(0, z)$  ▷ Numerical stabilization
  - 6:  $a \leftarrow \text{TR-exp}(-x_{\max})$
  - 7:  $b \leftarrow \text{TR-exp}(z - x_{\max})$
  - 8:  $s \leftarrow a + b$
  - 9:  $\ell_s \leftarrow \text{TR-exp}(-\text{TR-ln}(S))$
  - 10:  $\hat{\sigma} \leftarrow \ell_s \cdot a$  ▷  $\hat{\sigma} = \frac{a}{a+b}$  via log-domain division
  - 11:  $y \leftarrow x \cdot \hat{\sigma}$
  - 12: **return**  $y$
- 

### 3.2.3 Taylor Region Layer Normalization Approximation (TR-Norm)

The final nonlinear operation required for a fully-quantized ViT is LayerNorm. Introduced by Ba, et al. [24], LayerNorm rescales each feature vector according to:

$$\begin{aligned} \text{LayerNorm}(x_i) &= \gamma \frac{x_j - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \\ \mu &= \frac{1}{n} \sum_{j=1}^n x_j, \quad \sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_j - \mu)^2, \end{aligned} \quad (3.22)$$

where  $n$  is the vector size, and  $\gamma$  and  $\beta$  are trainable parameters. Because  $\mu$  and  $\sigma^2$  span a wide range, low-bit quantization is fragile. Consequently, most accelerators maintain

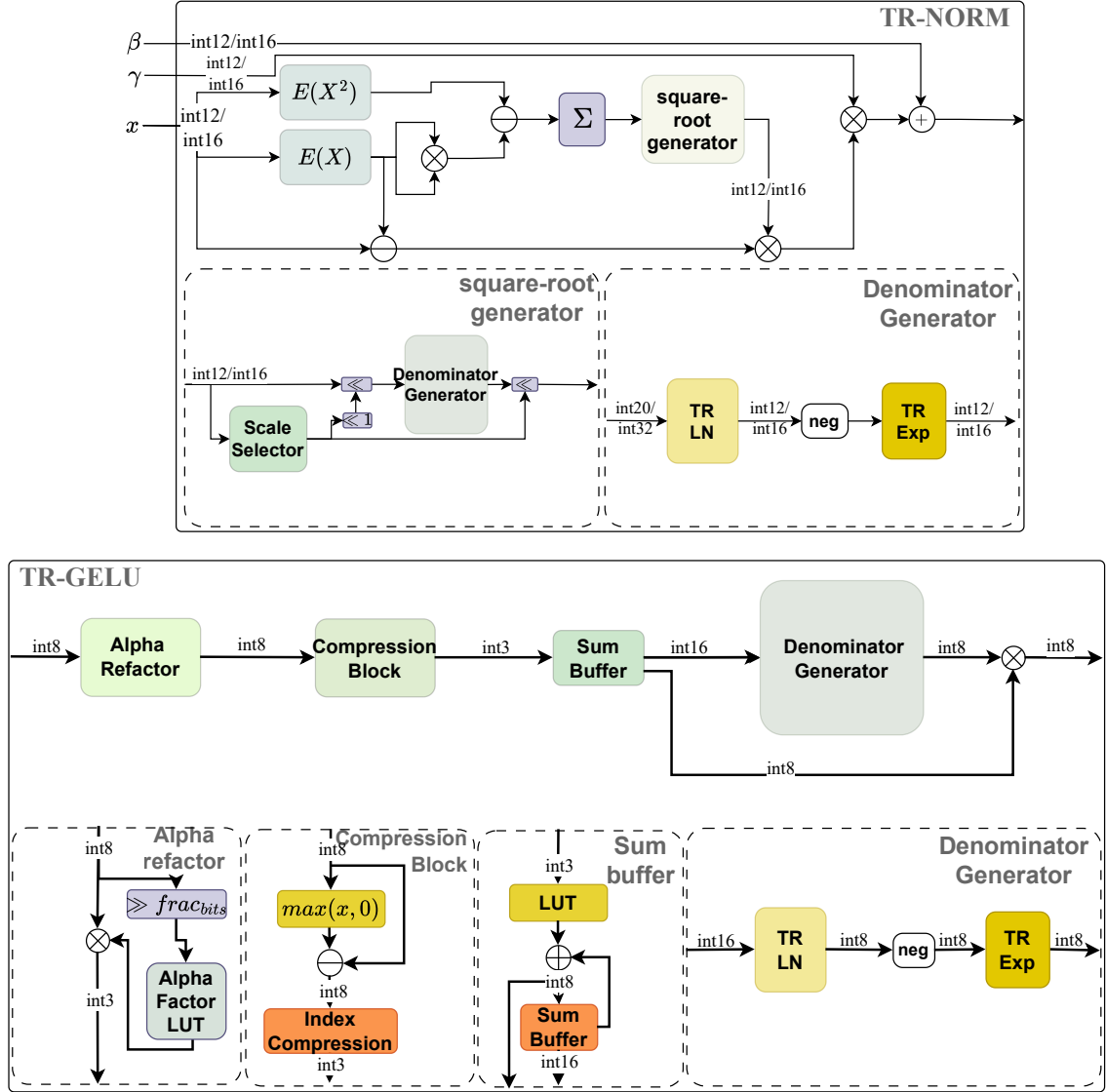


Figure 3.4: Block-level overview of the proposed TR-Norm (top) and TR-GELU (bottom) operators, illustrating their integration within the low-precision nonlinear processing pipeline.

a high-precision path via standalone dividers [8, 9, 21] or 32-bit “fake-FP32” blocks [1]. Iterative square-root ( $\sqrt{\cdot}$ ) schemes, such as Newton-Raphson [25], reduce the latency to only a few iterations, but still invoke a divider every cycle.

**Log-Domain Reformulation.** Similar to the approach for TR-SoftMax, we reduce the hardware requirements of the division and square-root by employing TR-exp and TR-ln:

$$\begin{aligned} \frac{1}{\sqrt{\sigma^2 + \epsilon}} &= \exp\left[\ln(\sigma^2 + \epsilon)^{-\frac{1}{2}}\right] \\ &\approx \text{TR-exp}\left[-\frac{1}{2} \text{TR-ln}(\sigma^2 + \epsilon)\right]. \end{aligned} \quad (3.23)$$

A difficulty emerges when  $\sigma^2 + \epsilon < 1$ , as the input to the logarithm becomes negative, resulting in reduced numerical fidelity in low-bit quantized formats. Addressing this region requires a computation mechanism whose precision can be selectively increased. To accommodate this, the exp LUT is expanded to include positive entries. Although this modification doubles the table size, it preserves the architectural structure and introduces only negligible hardware overhead. Fig. 3.4 provides the schemes for TR-GELU and TR-Norm. The proposed TR-Norm implementation is described in Algorithm 5.

---

**Algorithm 5** Taylor-Region LayerNorm Approximation (TR-Norm)

---

**Require:** input vector  $x = [x_1, \dots, x_n]$ , trainable parameters  $\gamma, \beta$ , constant  $\epsilon$

**Ensure:** normalized output  $y = [y_1, \dots, y_n]$

```

1:  $\mu \leftarrow \text{Mean}(x)$ 
2:  $x_{var} \leftarrow \text{Var}(x, \mu)$ 
▷ log-domain inverse square-root
3:  $\ell_v \leftarrow \text{TR-ln}(x_{var})$ 
4:  $\ell_r \leftarrow -\ell_v \gg 1$ 
5:  $r \leftarrow \text{TR-exp}(\ell_r)$ 
6: for  $j = 1$  to  $n$  do
7:    $\hat{x}_j \leftarrow x_j - \mu$ 
8:    $z_j \leftarrow \hat{x}_j \cdot r$ 
9:    $y_j \leftarrow \gamma \cdot z_j + \beta$ 
10: end for
11: return  $y$ 

```

---

**Outlier-Aware Scale Optimization.** As shown in Table 3.1, a significant source of accuracy degradation in low-precision LayerNorm arises from quantization of the scale parameter  $\gamma$ , whose distribution is highly skewed and outlier-dominated. In some configurations, this effect leads to severe accuracy loss, while in others it manifests as consistent two-digit percentage degradation. Direct min-max quantization therefore induces excessively wide dynamic ranges, collapsing most values into a small number of quantization levels. While prior work such as Quark [15] mitigates this issue via group-wise quantization, we adopt a complementary approach that optimizes a global scaling factor for

Table 3.1: Performance comparison (%) across GLUE tasks under different quantization configurations.

| CONFIG             | CoLA         | QNLI         | RTE          | MNLI         | QQP          | SST-2        | MRPC         |
|--------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| FP32 BASE          | 53.38        | 91.54        | 72.56        | 84.57        | 90.91        | 92.89        | 89.81        |
| ALL QUANT          | 51.27        | 88.30        | 70.76        | 81.94        | 90.07        | 92.78        | 88.43        |
| PIECEWISE          | 54.80        | 88.30        | 70.76        | 82.75        | 90.79        | 92.88        | 89.84        |
| EXCL LINEAR        | 55.91        | 90.99        | 71.48        | 83.48        | 90.79        | 93.23        | 90.97        |
| NORM (NO OPT)      | 2.95         | 65.17        | 55.23        | 53.48        | 70.55        | 86.01        | 38.75        |
| <b>NORM (OURS)</b> | <b>53.18</b> | <b>91.05</b> | <b>71.48</b> | <b>83.65</b> | <b>90.82</b> | <b>92.63</b> | <b>90.01</b> |

$\gamma$  using a lightweight line search. Specifically, we employ a variance-normalized Root Mean Square Error (RMSE) objective inspired by EasyQuant [26] to balance sensitivity to extreme outliers with accurate representation of typical values. The RMSE term penalizes large deviations caused by high-magnitude  $\gamma$  entries, while normalization by the tensor variance prevents degenerate solutions in which most elements collapse to a narrow dynamic range, yielding a stable and well-conditioned scale under aggressive quantization.

### 3.2.4 Gamma-like distribution of learned scales

A key challenge in the quantization of normalization layers arises from the distribution of the learned affine parameters  $\gamma$ . Unlike weights in convolutional or linear layers, which often follow near-Gaussian statistics, the scale parameters in transformer normalization layers are strictly positive and frequently heavy-tailed. As a result, a small number of large values can dominate the dynamic range.

Fig. 3.5 illustrates the effect of these outliers under naive min-max quantization. In several layers, the dynamic range is dictated by a few extreme values, causing the majority of  $\gamma$  parameters to collapse into the same quantization bin. This behavior significantly degrades the effective expressivity of the normalization layer.

### 3.2.5 Distribution-preserving scale optimization

**Variance-normalized line search.** Learned affine parameters ( $\gamma$ ) often exhibit heavy-tailed distributions in which large-magnitude values are not noise, but play a functional role in model expressivity. Standard clipping or percentile-based schemes are therefore suboptimal, as they may artificially compress the distribution and force pre-activations into a rigid unit-variance regime.

### 3.2. Hardware-Efficient TR Calculation of Nonlinear Deep Learning Operators

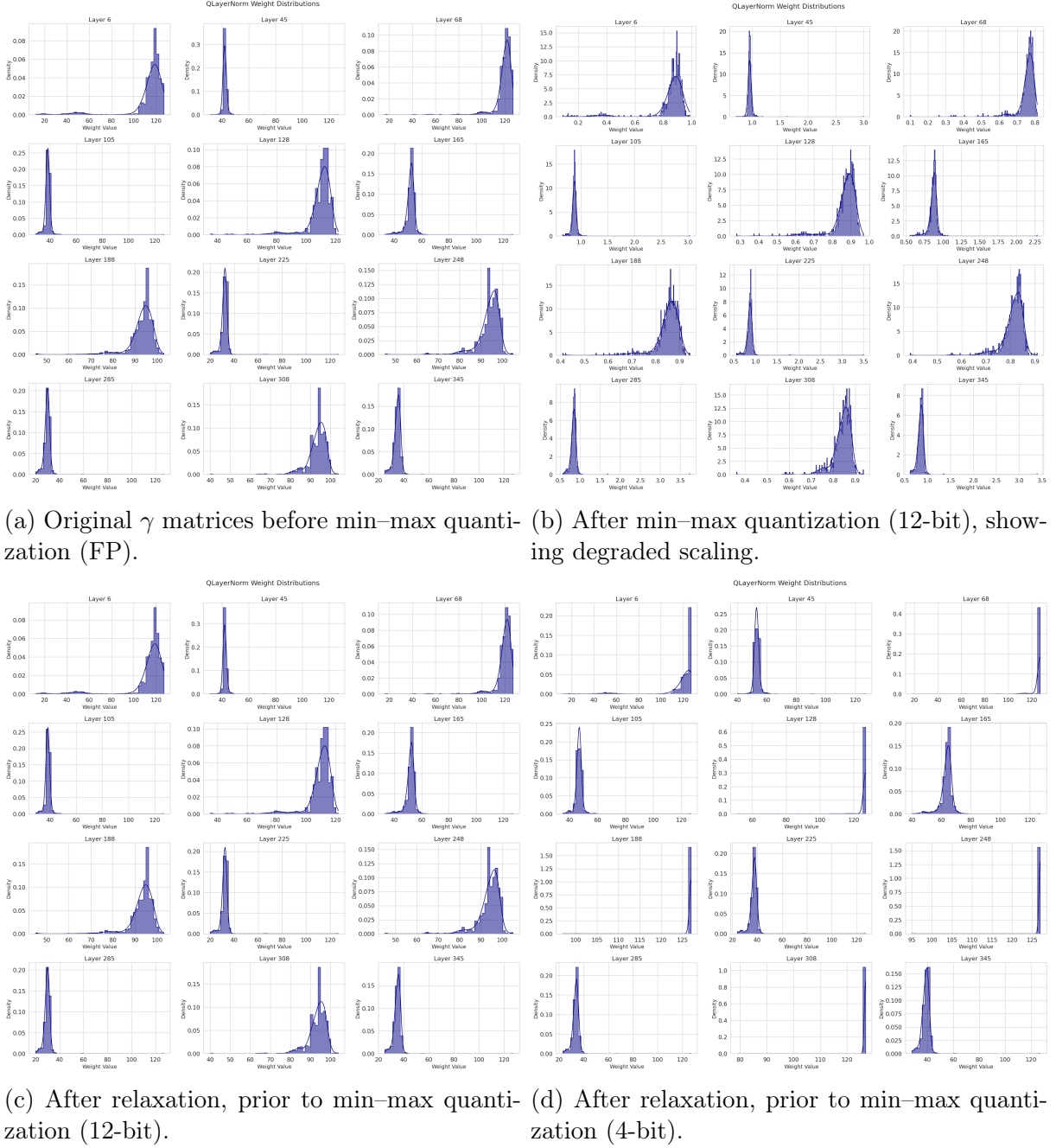


Figure 3.5: Effect of relaxation on LayerNorm  $\gamma$  parameters under min-max quantization. The top row shows the original  $\gamma$  matrices before quantization and the corresponding 12-bit min-max quantized result, where scale distortion is observed. The bottom row illustrates the same matrices after relaxation, prior to quantization: the relaxed parameters remain closer to the original, and even under a 4-bit representation exhibit improved stability compared to the unrelaxed 12-bit case.

To preserve the original distributional characteristics, we perform an offline line search for an optimal scaling factor  $s \in [0.5, 2.0]$  during calibration. For each affine layer, we select  $s$  by minimizing a variance-normalized RMSE objective:

$$\mathcal{L}(s) = \frac{\text{RMSE}(\gamma, \text{dequant}(\text{quant}(\gamma \cdot s)))}{\text{Var}(\gamma)}. \quad (3.24)$$

Normalizing the reconstruction error by the original parameter variance penalizes scales that would collapse the distribution. This objective favors quantized representations that retain the fluctuation structure of the FP32 baseline, thereby preserving the relative importance of high-magnitude channels.

# Chapter 4

## Experiments and Evaluation

### 4.1 Implementation Details

We implemented a fully-quantized, hardware-efficient ViT model, TR-ViT, utilizing the TR-exp, TR-ln, TR-SoftMax, TR-GELU, and TR-Norm primitives proposed in Chapter 3. TR-ViT is evaluated on widely adopted ViT architectures, including DeiT [19] and Swin Transformer [20], using the ImageNet-1k dataset [27]. To ensure a comprehensive analysis across varying model capacities, we test three configurations: tiny (T), small (S), and base (B). For post-training calibration, we sample 100 images from the training set and report accuracy on the full 50,000-image validation set.

All experiments were conducted on NVIDIA T4 GPUs. Our implementation utilizes Hugging Face libraries with streaming dataset support. While this may introduce minor latency, it significantly lowers the barrier for conducting large-scale experiments.

#### 4.1.1 Quantization Configuration

Per-tensor uniform quantization was applied across all major components. Linear projections and matrix multiplications are quantized to 8-bit precision for both inputs and weights.

The SoftMax (TR-SoftMax) and GELU (TR-GELU) computations, which require the evaluation of exponential and logarithmic functions, are implemented using 8-bit integer arithmetic. The specific precision requirements associated with the exponent across various layers are analyzed in the ablation studies (Section 4.2).

Layer normalization (TR-Norm) is realized using 12-bit arithmetic. This increased precision is necessary to maintain numerical stability during the computation of the mean, variance, and affine parameters ( $\gamma$  and  $\beta$ ). Although TR-Norm operates at a higher bit-width than other functional units, it eliminates the need for floating-point hardware while ensuring robust behavior across diverse model architectures.

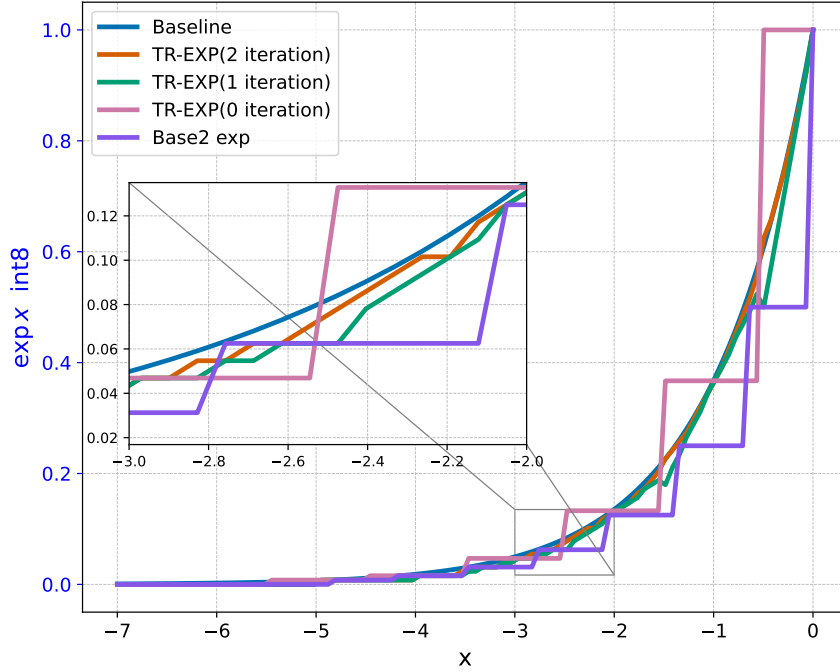


Figure 4.1: Comparison of exponential function through different orders of polynomial approximations and log based approximation.

### 4.1.2 Core Error Analysis

The first-order TR-Exp approximation exhibits bounded absolute error across the evaluated ranges, as illustrated in Fig. 4.1. For the SoftMax regime ( $x \in [-7, 0]$ ) under 8-bit fixed-point representation, the worst-case absolute error remains below approximately 0.4. Higher-precision configurations, such as the 12-bit evaluation over  $x \in [0, 4]$  shown in Fig. 4.4, reduce the error to within roughly 0.1. Since the exponential outputs are subsequently normalized (e.g., through  $1/\sum \exp(\cdot)$ ), these errors are further attenuated in practice. Consequently, the approximation remains sufficiently accurate for downstream log-domain division and inverse square-root computations.

The TR-Ln approximation maintains bounded error of approximately  $\pm 0.125$  across the evaluated dynamic range, demonstrating stable behavior even for large inputs. Notably, the proposed formulation replaces conventional double-width division with simple shift and multiplication operations, enabling both division and inverse square root using only two same-width multiplications.

### 4.1.3 Accuracy Evaluation

TR-ViT is benchmarked against the full-precision (FP32) baseline and recent state-of-the-art PTQ methods, including FQ-ViT [21], SOLE [10], QUARK [15], and the method proposed by Zhang et al. [22], which integrates I-BERT’s integer GELU with a constrained



Table 4.1: Comparison of Top-1 accuracy (%) across different ViT models and quantization methods.

| METHOD               | DEiT-T       | DEiT-S       | DEiT-B       | SWIN-T       | SWIN-S       | SWIN-B             |
|----------------------|--------------|--------------|--------------|--------------|--------------|--------------------|
| FULL FP32 (BASELINE) | 72.21        | 79.85        | 81.85        | 81.35        | 83.20        | 83.60              |
| FQ-ViT [21]          | 71.61        | 79.17        | 81.20        | 80.51        | 82.71        | 82.97              |
| ZHANG [22]           | 71.08        | 78.49        | 80.74        | 80.03        | 82.29        | 82.67              |
| SOLE (INT8)          | 71.07        | 78.89        | 81.12        | 80.14        | 82.60        | 82.79              |
| QUARK [15]           | 71.29        | 79.40        | 81.52        | 81.06        | 82.81        | 85.03 <sup>†</sup> |
| <b>TR-ViT (OURS)</b> | <b>71.39</b> | <b>79.27</b> | <b>81.40</b> | <b>80.70</b> | <b>82.80</b> | <b>83.18</b>       |

<sup>†</sup>QUARK reports 85.27% accuracy on Swin-B when evaluated against its own FP32 baseline.

quantization scheme.

Table 4.1 summarizes the Top-1 accuracy on ImageNet. Our method is among the top performer across all configurations. Compared to the FP32 baseline, the accuracy degradation remains less than 1% for the majority of models, a notable achievement given the aggressive integer-only quantization applied to all nonlinear layers.

## 4.2 Ablation Study

To understand the effect of various design decisions on TR-ViT, we conduct ablation studies across three axes: (1) SoftMax approximation, (2) GELU quantization, and (3) LayerNorm precision.

### 4.2.1 SoftMax Robustness

Replacing the full-precision SoftMax operation with the proposed 8-bit TR-SoftMax approximation results in less than 1% accuracy degradation on vision tasks and modest relative degradation across GLUE benchmarks. The maximum relative drop remains below 4%, with most tasks exhibiting less than 3% degradation that causes by the linear layers.

Despite the widespread perception that exponential functions are highly sensitive to approximation, we find that the SoftMax operation is surprisingly robust to quantization noise. This robustness stems from two key factors. First, the normalization step rescales each exponential relative to the sum, thereby attenuating the impact of individual approximation errors. Second, the attention mechanism primarily relies on preserving the ordering and relative magnitude of dominant logits rather than exact probability values; contributions from strongly negative logits (e.g.,  $e^{-8} \approx 3 \times 10^{-4}$ ) are negligible and can be coarsely approximated without affecting the output.

Table 4.2: Model Accuracy Across Different Bit-Precisions

| METHOD                   | DeiT-T       | DeiT-S       | DeiT-B       | SWIN-T       | SWIN-S       | SWIN-B       |
|--------------------------|--------------|--------------|--------------|--------------|--------------|--------------|
| FP32 (BASELINE)          | 72.21        | 79.85        | 81.85        | 81.35        | 83.20        | 83.60        |
| 4-BIT (ROUNDING)         | 68.11        | 77.21        | 80.53        | 79.23        | 80.81        | 81.28        |
| 6-BIT (ROUNDING)         | 70.64        | 78.61        | 81.29        | 80.28        | 82.67        | 82.97        |
| 8-BIT (ROUNDING)         | 70.84        | 78.50        | 81.21        | 80.34        | 82.73        | 82.98        |
| <b>8-BIT (1ST ORDER)</b> | <b>71.39</b> | <b>79.27</b> | <b>81.40</b> | <b>80.70</b> | <b>82.80</b> | <b>83.18</b> |

To further stress the design, we evaluate increasingly aggressive exponential approximations. Using a zero-order approximation in Q4.4 format (comprising 1-bit sign, 3-bit integer, and 4-bit fractional components) has minimal impact on accuracy. This suggests that the attention mechanism is largely invariant to the exact precision of the exponential tail, provided the relative order of magnitudes is preserved. In contrast, the max-subtraction stage ( $\hat{x} = x - x_{\max}$ ) is significantly more sensitive to quantization noise. As shown in Table 4.2, reducing the precision of this stage to 4–6 bits leads to larger degradation, particularly in compact architectures like DeiT-T. This sensitivity arises because max-subtraction acts as a global shift; errors in  $x_{\max}$  propagate to every element in the SoftMax input, potentially shifting the distribution into regions where the Taylor approximation is less accurate or where the exponential values saturate near zero. These results indicate that performance degradation is primarily driven by max-subtraction, rather than by inaccuracies in the exponential approximation itself.

### Preprocessing and numerical stabilization.

Given an input vector  $x$ , we compute the per-configuration maximum  $x_{\max} = \max(x)$  and form the stabilized vector  $x' = x - x_{\max}$  so that all entries are non-positive. Importantly, the max-reduction and subtraction are performed *in the target bit-precision* (8-bit, 6-bit, or 4-bit, respectively), using the corresponding  $Q_{I,K}$  format. The stabilized values  $x'$  are then used as inputs to the exponential evaluation.

**Zero-order exponential via a compact LUT.** To stress-test robustness, we evaluate a zero-order approximation in which the fractional bits are discarded (i.e., we keep only the integer component of  $x'$ ). The exponential is then computed using a  $2^I$ -entry lookup table indexed by the integer value (e.g., 16 entries for  $I = 4$ ):

$$\text{LUT}[a] = \text{round}(e^a \cdot (2^{b-1} - 1)),$$

where  $b$  is the output bit-width. Values with  $a < -6$  are clamped to zero to avoid underflow. This provides a lightweight alternative to implementing a full 8-bit exponential table (256 entries), while keeping the evaluation hardware-efficient.

**Accuracy vs. bit precision.** We use signed two’s-complement fixed-point numbers in the  $Q_{I,K}$  convention, where  $I$  counts integer bits *including the sign bit* and  $K$  counts fractional bits (total width  $I+K$ ). For SoftMax, we apply standard numerical stabilization via max-subtraction,  $x' = x - x_{\max}$  with  $x_{\max} = \max(x)$ , which enforces  $x' \leq 0$ . Hence, although  $Q_{4.4}$  nominally spans  $[-8, 7.9375]$ , only the non-positive half is exercised in practice, i.e.,  $x' \in [-8, 0]$ . Equivalently, the integer field contains one sign bit and three magnitude integer bits. In the Taylor-Region (TR) formulation, the integer component selects the region (evaluation point), while the fractional component is the local residual used by higher-order terms. Our primary configurations are  $Q_{4.4}$  (8-bit),  $Q_{4.2}$  (6-bit), and  $Q_{4.0}$  (4-bit).

Table 4.2 reports Top-1 accuracy across DeiT and Swin models. Relative to FP32, 4-bit rounding exhibits the largest degradation, with a drop of 1.32–4.10 points (median 2.35). In contrast, 6-bit and 8-bit rounding reduce the loss to 0.53–1.57 (median 0.85) and 0.47–1.37 (median 0.83) points, respectively. 1st order configuration further improves accuracy, limiting the drop to 0.40–0.82 points (median 0.52).

**Hardware-oriented execution flows.** Based on the accuracy–precision trade-off observed above, we propose two alternative hardware flows for implementing the exponential operator within SoftMax as shown in Fig. 4.3 and Fig. 4.2.

In the zero-order TR configuration, the exponential is approximated using only the region anchor  $e^a$  selected by the integer field. Instead of storing full 8-bit exponent values for each activation, we store only the 3-bit region index corresponding to the LUT entry. This significantly reduces memory footprint and bandwidth. During matrix multiplication with the value tensor, the compressed representation is decompressed on-the-fly by fetching the corresponding anchor value from the LUT and applying it within the multiply–accumulate (MAC) datapath. Since the integer component already determines the TR region, this approach preserves the structural formulation while minimizing storage cost.

For higher accuracy, the first-order TR formulation can be employed. Rather than computing  $\exp(x)$  in a dedicated exponential unit, we decompose it into the product of the anchor term  $e^a$  and the local linear correction  $(1 + \Delta)$ . These components are then injected directly into the multiplication unit. This allows the exponential scaling to be fused into the matrix multiplication stage, exploiting the inherent parallelism of

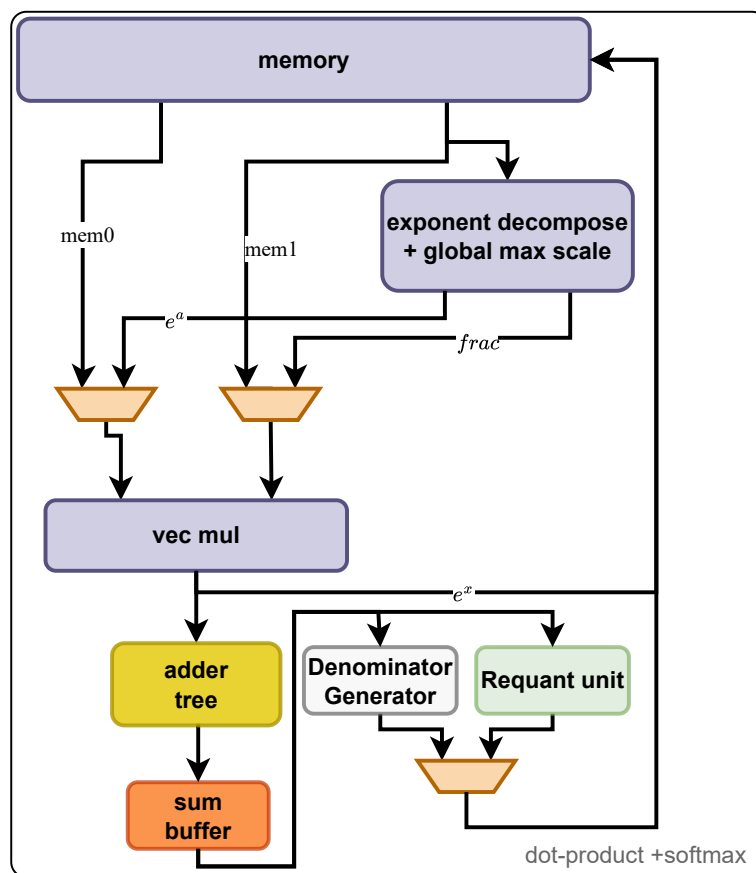


Figure 4.2: Block-level overview of combined first-order exponent and dot product unit.

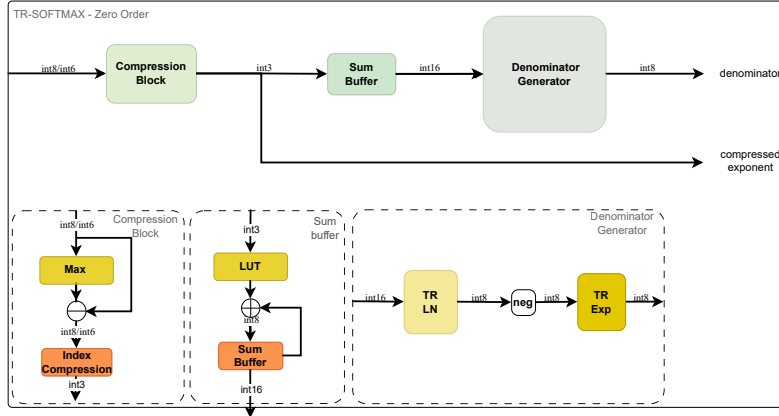


Figure 4.3: Block-level overview of compressed post exponent values.

Table 4.3: Impact of GELU implementation on model accuracy. Only the GELU nonlinearity is quantized; all other operations remain in floating point.

| CONFIGURATION                                         | ACCURACY (%) |
|-------------------------------------------------------|--------------|
| FP32 BASELINE (ALL LAYERS)                            | 72.21        |
| INTEGER GELU ( $\alpha = 1.702$ )                     | 71.07        |
| <b>INTEGER GELU (SEGMENTED <math>\alpha_i</math>)</b> | <b>72.13</b> |

the MAC array with minimal additional logic. By eliminating a separate nonlinear unit and reusing existing multiplier resources, this flow reduces control overhead and improves datapath efficiency while maintaining improved numerical fidelity relative to the zero-order configuration.

### 4.2.2 GELU Optimization

Our design choice to adopt the proposed compression was inspired by the structural similarity between the attention mechanism and the feed-forward (MLP) layers, both of which function as key-value memory systems. Based on our finding that attention does not require high-precision representations, we hypothesized that a similar compression strategy would apply to MLP layers.

We push this idea further by evaluating GELU under an aggressive mixed-precision configuration: activation inputs and outputs use 8 bits, while the exponential used in the sigmoid approximation is computed using only 4-bit precision (denoted as 8/4/8). This setting represents the lowest-precision exponential approximation considered in our study.

Results indicate that smaller models are particularly sensitive. For example, on DeiT with this precision and a global scaling factor  $\alpha = 1.702$ , the accuracy degradation is ap-

Table 4.4: Vision model performance (Top-1 Acc. %) under 4-bit weight and 12-bit activation quantization.

| <b>Model</b>    | DeiT-T | DeiT-S | DeiT-B | Swin-T | Swin-S | Swin-B |
|-----------------|--------|--------|--------|--------|--------|--------|
| <b>Acc. (%)</b> | 70.71  | 78.67  | 81.03  | 78.48  | 82.60  | 82.79  |

Table 4.5: GLUE benchmark results under 4-bit weight and 12-bit activation quantization.

| <b>Task</b>   | CoLA  | SST-2 | QQP   | MNLI  | QNLI  | RTE   |
|---------------|-------|-------|-------|-------|-------|-------|
| <b>Metric</b> | 50.73 | 92.66 | 89.71 | 83.03 | 88.14 | 71.84 |

proximately 1.15%. In this regime, GELU becomes the dominant source of error. However, the proposed region-based scaling recovers nearly all of this residual loss, demonstrating that fine-grained control of the activation function is necessary to fully exploit aggressive quantization, as shown in Table 4.3.

### 4.2.3 Layer Normalization Analysis

Across the operational range, the proposed log-domain Taylor approximation remains stable and accurate, with precision controlled by a small number of TR-exp iterations (at most two in all evaluated configurations), yielding a favorable accuracy-cost trade-off. Although worst-case degradation under full quantization is observed in Table 3.1, ablation analysis indicates that it is primarily driven by the linear layers. In contrast, per-tensor quantization of nonlinear operators introduces only marginal degradation, while the dominant source of error in LayerNorm arises from the scale parameter  $\gamma$ , motivating our  $\gamma$ -specific treatment.

**Empirical search range.** Across all evaluated models, the optimal scale consistently lies within the  $[0.5, 2.0]$  interval. Restricting the search to this range allows the TR-PTQ pipeline to adapt to the distinct statistical profiles of different architectures (e.g., higher variance in DeiT versus more localized scales in Swin) without per-layer heuristics or training-aware fine-tuning.

**Mitigating gamma-induced degradation.** To assess the sensitivity of the affine parameters, we conduct an ablation in which  $\gamma$  is quantized to 4-bit precision while all other parameters remain in floating point. Under naive min-max quantization, the heavy-tailed distribution of  $\gamma$  leads to severe resolution loss for most channels, resulting in a dramatic degradation in model accuracy.

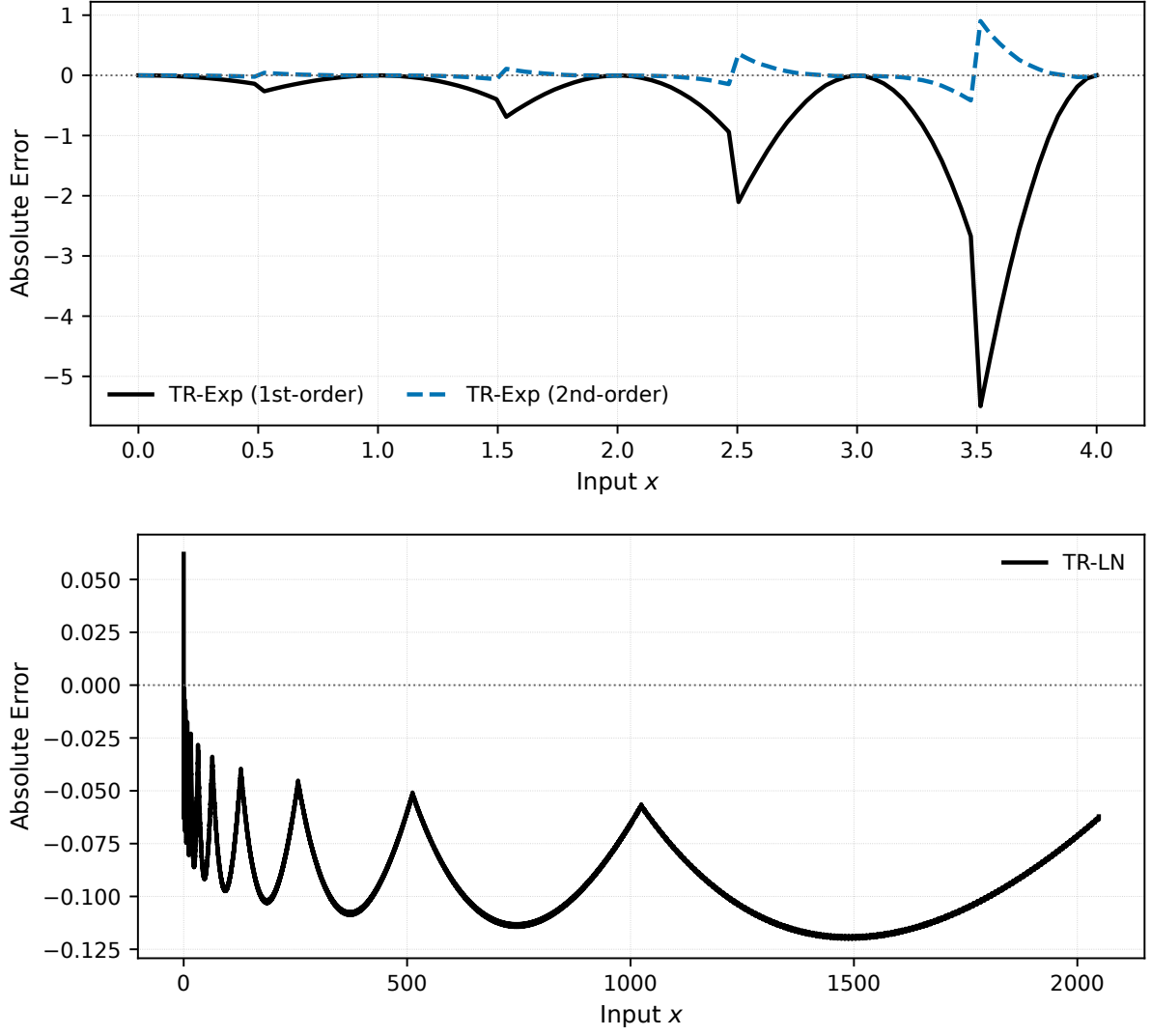


Figure 4.4: Approximation error of the proposed TR nonlinear operators (top: TR-exp, bottom: TR-ln) under 12-bit integer arithmetic.

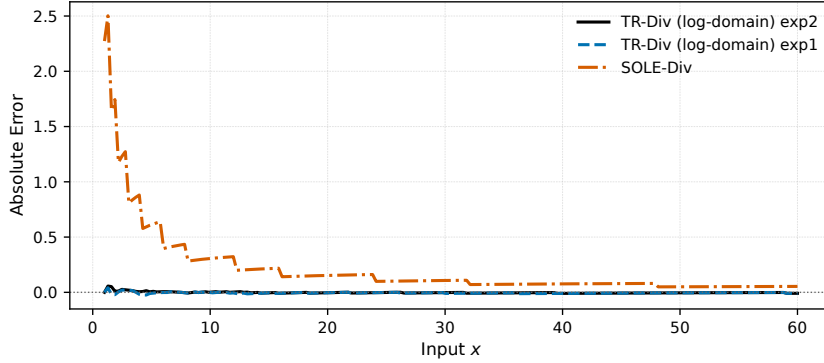


Figure 4.5: Comparison of division noise between the proposed method and a low-cost comparable solution (SOLE). The plot reports the absolute error with respect to a high-precision reference across the evaluated input range.

Applying the proposed distribution-preserving line search restores functional behavior. As reported in Tables 4.5 and 4.4, the optimized models recover to within 1–2% of their FP32 baselines. For example, Swin-T recovers to 78.48% Top-1 accuracy. This result indicates that scale optimization is not merely a refinement step, but a prerequisite for stable low-precision quantization of normalization layers in integer-only Transformer models.

**Numerical behavior of TR-Div and TR-Norm.** The evaluation of TR-Div remains numerically stable under the considered Transformer workloads. Since the denominator is constrained to be  $\geq 1$  - either through the addition of a stabilization constant  $\epsilon$  or by properties of the attention score normalization-the corresponding logarithm is non-negative. As a result, division in the log domain yields values bounded by unity, which in turn constrains the subsequent exponential evaluation to a limited dynamic range. This bounded regime is well suited to low-order Taylor approximations; empirically, a second-order polynomial provides the best accuracy-complexity trade-off.

Fig. 4.5 compares TR-Div against a shift-based division method [10]. The results indicate that TR-Div exhibits lower numerical error across the evaluated ranges, which we attribute to the avoidance of coarse quantization effects introduced by bit-shift operations.

In contrast, TR-Norm is more sensitive to quantization due to its broader dynamic range. Unlike TR-Div, the logarithmic inputs encountered in normalization layers may take both positive and negative values, requiring a wider representation to preserve accuracy. We therefore adopt a 12-bit configuration as a balance between numerical fidelity and implementation cost. Importantly, the second-order Taylor term  $(x - a)^2/2$  depends only on the fractional residual, enabling its computation with a 12-bit multiplier. The anchor-point exponential  $e^a$  is pre-computed and absorbed into the layer-specific scal-



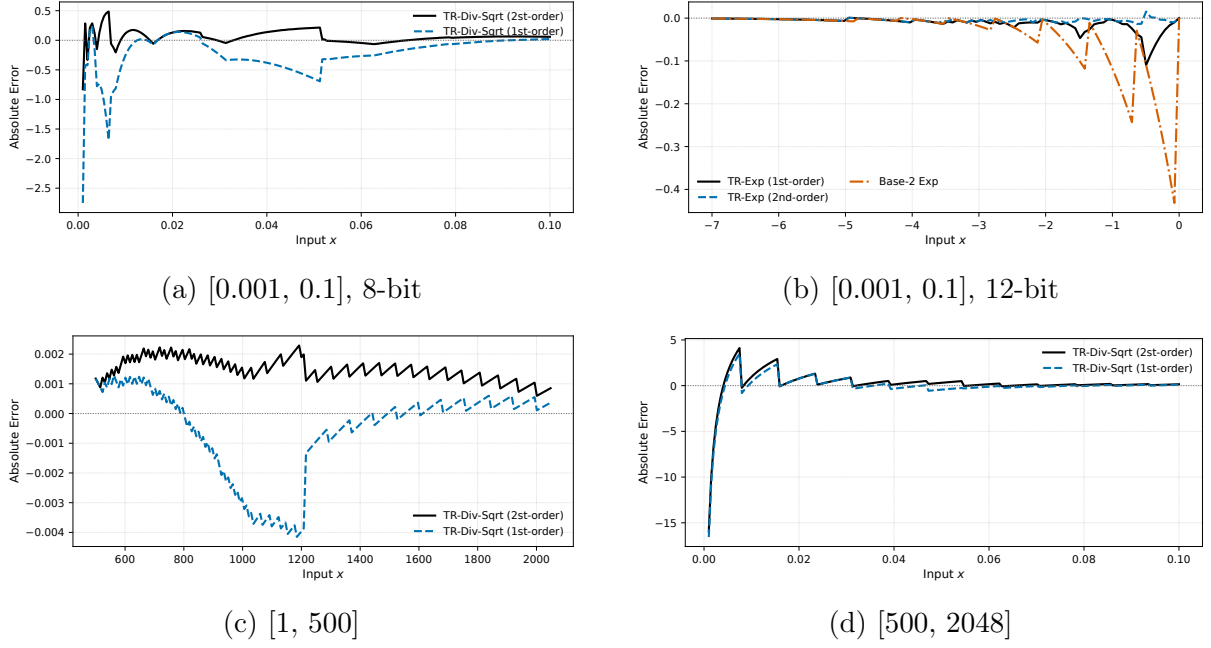


Figure 4.6: Division noise comparison across different input ranges and bit-precisions. Noise is measured as absolute error with respect to a high-precision reference.

ing factor during calibration, reducing the inference-time computation to a single 12-bit multiplication. Fig. 4.6 show Division noise comparison across different input ranges.

Since the multiplier constitutes the primary arithmetic bottleneck, this fusion enables both division and square-root operations to be implemented within a single computation cycle. The 12-bit intermediate precision is used only for internal accumulation to prevent error propagation, while the final output is re-quantized to 8-bit before being forwarded to subsequent layers.



## Chapter 5

### Summary and Conclusions

This thesis addressed the challenge of enabling efficient integer-only quantization for transformer-based models, with the goal of reducing their computational and memory footprint while preserving accuracy. We presented a framework that approximates critical non-linear operations – including SoftMax, GELU, and LayerNorm – using lightweight Taylor-series expansions and small LUT-based designs. Our approach avoids the need for costly operations such as division and square root, instead relying on compact arithmetic units that are well suited for hardware deployment. A series of ablation studies across both vision and language tasks demonstrated that the proposed approximations preserve accuracy, with minimal degradation even under aggressive low-bit quantization. In particular, we highlighted that the normalization step and division operations play a more decisive role in accuracy than the exponential itself, while controllable precision allows selective refinement of different components.

Beyond methodological contributions, this work provides practical insights into the trade-offs between bit precision, hardware complexity, and model performance. By validating our methods on representative architectures such as BERT and DeiT, we demonstrated that integer-only transformers can achieve competitive accuracy without resorting to mixed-precision or floating-point computation.

Looking forward, we intend to release our codebase as an open-source project on GitHub, making our framework accessible to both the research community and practitioners. Furthermore, we are planning a hardware implementation to validate the efficiency of our approach on dedicated accelerators. Together, these efforts lay the groundwork for deploying state-of-the-art transformer models in resource-constrained environments, bridging the gap between theoretical advances in quantization and their practical realization in hardware.

To ensure full transparency and reproducibility, all experimental setups, quantization procedures, and hardware-oriented implementations are publicly available. The complete source code and evaluation scripts can be accessed at:

`https://github.com/enics-labs/TR-PTQ.git`

The repository contains the necessary instructions to reproduce all reported results.

# Bibliography

- [1] J. Zhang, H. Liu, Y. Zhang, M. Li, and Z. Zhan, “Faster Transformer-DS: Multiscale Vehicle Detection of Remote-Sensing Images Based on Transformer and Distance-Scale Loss,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 17, pp. 1961–1975, 2024.
- [2] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 38 087–38 099. [Online]. Available: <https://proceedings.mlr.press/v202/xiao23c.html>
- [3] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “Awq: Activation-aware weight quantization for on-device llm compression and acceleration,” in *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., vol. 6, 2024, pp. 87–100. [Online]. Available: [https://proceedings.mlsys.org/paper\\_files/paper/2024/file/42a452cbafa9dd64e9ba4aa95cc1ef21-Paper-Conference.pdf](https://proceedings.mlsys.org/paper_files/paper/2024/file/42a452cbafa9dd64e9ba4aa95cc1ef21-Paper-Conference.pdf)
- [4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.17323>
- [5] J. Yu, J. Park, S. Park, M. Kim, S. Lee, D. H. Lee, and J. Choi, “NN-LUT: neural approximation of non-linear operations for efficient transformer inference,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 577–582. [Online]. Available: <https://doi.org/10.1145/3489517.3530505>
- [6] W. Zhong, H. Feng, Z. Zhou, H. Peng, and S. Yu, “Intattention: A fully integer attention pipeline for efficient edge inference,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.21513>

- [7] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, “Softermax: Hardware/Software Co-Design of an Efficient Softmax for Transformers,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 469–474.
- [8] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, “I-BERT: integer-only BERT quantization,” *CoRR*, vol. abs/2101.01321, 2021. [Online]. Available: <https://arxiv.org/abs/2101.01321>
- [9] Z. Li and Q. Gu, “I-ViT: Integer-only quantization for efficient vision transformer inference,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 17 065–17 075.
- [10] W. Wang, S. Zhou, W. Sun, P. Sun, and Y. Liu, “Sole: Hardware-software co-design of softmax and layernorm for efficient transformer inference,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [11] J. N. Mitchell, “Computer multiplication and division using binary logarithms,” *IRE Transactions on Electronic Computers*, vol. EC-11, no. 4, pp. 512–517, 1962.
- [12] S. Kim and C. E. Rhee, “Arnorm: Hardware-efficient normalization for lightweight edge models,” in *2025 International Technical Conference on Circuits/Systems, Computers, and Communications (ITC-CSCC)*, 2025, pp. 1–3.
- [13] J. Park and C. E. Rhee, “High-precision softmax division without multipliers or look-up tables,” in *2025 International Technical Conference on Circuits/Systems, Computers, and Communications (ITC-CSCC)*, 2025, pp. 1–3.
- [14] A. Marchisio, D. Dura, M. Capra, M. Martina, G. Masera, and M. Shafique, “Swift-Tron: An Efficient Hardware Accelerator for Quantized Transformers,” in *2023 International Joint Conference on Neural Networks (IJCNN)*, 2023, pp. 1–9.
- [15] Z. Zhao, H. Li, F. Liu, Y. Lu, Z. Wang, T. Yang, L. Jiang, and H. Guan, “Quark: Quantization-enabled circuit sharing for transformer acceleration by exploiting common patterns in nonlinear operations,” in *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2025, pp. 1–9.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.

- 
- [17] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
- [18] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pp. 4171–4186.
- [19] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training Data-Efficient Image Transformers & Distillation Through Attention,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 10 347–10 357.
- [20] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 10 012–10 022.
- [21] Y. Lin, T. Zhang, P. Sun, Z. Li, and S. Zhou, “FQ-ViT: Post-training quantization for fully quantized vision transformer,” *arXiv preprint arXiv:2111.13824*, 2021.
- [22] Z. Zhang, B. He, and Z. Zhang, “Practical edge kernels for integer-only vision transformers under post-training quantization,” *Proceedings of Machine Learning and Systems*, vol. 5, pp. 35–47, 2023.
- [23] D. Hendrycks and K. Gimpel, “Bridging nonlinearities and stochastic regularizers with gaussian error linear units,” *CoRR*, vol. abs/1606.08415, 2016. [Online]. Available: <http://arxiv.org/abs/1606.08415>
- [24] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [25] R. Crandall and C. Pomerance, “Prime numbers: A computational perspective,” 2001.
- [26] D. Wu, Q. Tang, Y. Zhao, M. Zhang, Y. Fu, and D. Zhang, “Easyquant: Post-training quantization via scale optimization,” 2020. [Online]. Available: <https://arxiv.org/abs/2006.16669>

- [27] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *2009 IEEE conference on computer vision and pattern recognition*. IEEE, 2009, pp. 248–255.



# תקציר

ארכיטקטורת הטרנספורמר (transformer) הפכה בשנים האחרונות לאבן יסוד במערכות למידת מכונה מודרניות, ומשיגה ביצועים מובילים במגוון משימות בראייה ממוחשבת ובעיבוד שפה טבעית. עם זאת, המורכבות החישובית הגבוהה ודרישות הזיכרון המאסיביות של מודלים אלה מציבות אתגר משמעותי בפריסתן על גבי מערכות משובצות (embedded systems) ומאצי חומרה ייעודיים, הפועלים תחת מגבלות של שטח סיליקון וצריכת אנרגיה. אחת הגישות המרכזיות להתמודדות עם מגבלות אלו היא קוונטיזציה, ובמיוחד קוונטיזציה למודל מאומן (PTQ, Post-Training Quantization), המאפשרת להמיר מודלים מאומנים בייצוג נקודה (Floating-point) צפה לייצוגים מדויקים פחות אך יעילים יותר חישובית, ללא צורך באימון מחדש.

למרות ההצלחתה של שיטה זו ברשתות קונבולוציוניות, מודלי טרנספורמר מתגלים כרגישים במיוחד לירידה בדיוק תחת קוונטיזציה, בעיקר כאשר קיימת דרישה למימוש באמצעות חישוב שלם בלבד וברמת דיוק נמוכה. רגישות זו מיוחסת לרוב לשכבות הלא-ליניאריות במודל, כגון פונקציות SoftMax ו-GELU ושכבת הנרמול LayerNorm. כתוצאה מכך, מודלים רבים משאירים רכיבים אלו בחישובי בתצורה המקורית נקודה צפה או משתמשות בשיטות קירוב איטרטיביות אשר מגדילות את מורכבות החומרה ואת עלות המימוש.

בעבודה זו אנו מציעים את TR-PTQ, מסגרת קוונטיזציה למודל מאומן המבוססת על רפורמולציה של פונקציות לא-ליניאריות במרחב הלוגריתמי. בגישה זו פעולות כפל, חילוק וחישוב שורש ריבועי מומרות לפעולות חיבור והזזה במרחב הלוגריתמי, וכך ניתן לממשן באמצעות אריתמטיקה שלמה בייצוג משלים ל-2. רפורמולציה זו מבטלת את הצורך ביחידות חומרה ייעודיות לחילוק ולשורש ריבועי ומייתרת שימוש בשיטות חישוב איטרטיביות, תוך שמירה על שגיאת קירוב מבוקרת.

המסגרת המוצעת מאחדת את אבני הבניין של אקספוננט ולוגריתם עבור פונקציות SoftMax, GELU ו-LayerNorm באמצעות קירוב פולינומיאלי לאקספוננט תוך שימוש בטבלאות בגודל מינימלי ומימוש תת פולינומיאלי ללוגריתם הטבעי. אבני בניין אלה מאפשרות מסלול חישוב אחיד ודטרמיניסטי לכל האי-ליניאריות במודל, מפחיתות כפילות במימוש החומרה ומבטלות את ההסתמכות על חישובי נקודה צפה. בנוסף, אנו מציעים אופטימיזציה מודעת לערכי קיצון עבור פרמטרי LayerNorm בשלב הקליברציה, המקטינה את הגברת הטווח הדינמי מבלי להזדקק לאימון מחדש או לביצוע חישובים במדויקויות מעורבות.

אנו גם מראים כי הירידה בביצועים אינה נובעת בעיקר ממגבלת הדיוק הנומרי של הייצוג המספרי, אלא ממנגנוני הגברת שגיאה מבניים הנוצרים בשילוב בין שכבות שונות במודל. הניתוח שלנו מצביע על שני גורמים מרכזיים. הראשון הוא פרמטרי הסקייל הנלמדים בשכבות הנרמול, אשר עלולים להגביר רעש קוונטיזציה בשל שונות בין-ערוצית גבוהה והתרחבות טווח הערכים הדינמי. השני הוא הצטברות שגיאות קירוב בפונקציית GELU כאשר אופרטורים לא-ליניאריים מקורבים באופן בלתי תלוי בייצוג נקודה קבועה. לעומת זאת, נמצא כי פונקציית SoftMax שומרת על יציבות יחסית גם תחת קוונטיזציה אגרסיבית, כל עוד נשמרת יציבות נומרית באמצעות חיסור הערך המקסימלי.

אנו מעריכים את TR-PTQ על מודלי טרנספורמר מייצגים במשימות ראייה ממוחשבת ועיבוד שפה טבעית תחת קוונטיזציה של 8 ביט. התוצאות מראות כי השיטה שומרת על ביצועים הקרובים למודל המקורי, עם ירידת דיוק מוחלטת של פחות מ-1.5% ביחס לבסיס בנקודה צפה במדדי הבנצ'מרק המקובלים. ממצאים אלו מדגימים כי רפורמולציה מבנית במרחב הלוגריתמי, המותאמת מראש לאילוצי חומרה, מאפשרת הסקה יציבה של מודלי טרנספורמר באמצעות חישוב שלם בלבד.

גרסה של עבודה זו הוגשה לכנס ICML ונמצאת כעת בתהליך ביקורת עמיתים.

עבודה זו נעשתה בהדרכתו של פרופסור אדם תימן  
מהפקולטה להנדסה של אוניברסיטת בר-אילן

אוניברסיטת בר-אילן

TR-PTQ : קוונטיזציה של טרנספורמרים במרחב  
הלוגריתמי בייצוג שלם למודל מאומן באמצעות  
קירובי טיילור לפי סגמנטים  
אליהו לוי

עבודה זו מוגשת כחלק מהדרישות לשם קבלת תואר מוסמך  
בפקולטה להנדסה של אוניברסיטת בר אילן

שבט, תשפ"ו

רמת גן